

ENTORNOS DE DESARROLLO

APP de gestión de incidencias

App en Python con Streamlit

e Interfaz Gráfica

Arquitectura de Capas

Versiones V2 y V3

Alumno: Francisco José García Franco

1º DAM - UAX

2025/2026

APP PARA GESTIÓN DE INCIDENCIAS V2 y V3

AMPLIACIÓN – APP CON INTERFAZ GRÁFICA EMPLEANDO STREAMLIT

1. OBJETIVO

2. VERSIÓN 2 – APP CON INTERFAZ GRÁFICA

2.1 CÓDIGO

- [Consideraciones previas](#)
- [Esquema básico de la nueva versión app / mapa de interconexión de archivos](#)
- [Código de la interfaz](#)
 - [Interfaz – Archivo .py:](#) [gestion incidencias.py](#)
 - [Interfaz – Archivo .css:](#) [styles.css](#)
 - [Modificaciones en backend:](#)
 - [Archivo gestor de la BBDD:](#) [db.py](#)
 - [Repositorio \(operaciones SQL\):](#) [repository.py](#)
 - [Servicio:](#) [service.py](#)

2.2 EJECUCIÓN EN TERMINAL DE LA APLICACIÓN CON INTERFAZ GRÁFICA

- [Puesta en funcionamiento de la app](#)
- [Aspecto de la interfaz y pruebas funcionales](#)
 - [Aspecto inicial con la base de datos aún no creada](#)
 - [Proceso de creación de tickets](#)
 - [Proceso de creación de técnicos](#)
 - [Asignación de una incidencia a un técnico](#)
 - [Cierre de una incidencia](#)
 - [Desactivación de un técnico](#)
 - [Gestión de Reglas de Negocio no permitidas](#)

3. VERSIÓN 3 – VERSIÓN EJECUTABLE EN WINDOWS DE LA APP CON INTERFAZ GRÁFICA

3.1 PROCEDIMIENTO DE COMPILACIÓN

- [Consideraciones previas](#)
- [Obstáculos e Hitos en el procedimiento de empaquetado](#)

3.2 CÓDIGO DE LOS NUEVOS ARCHIVOS PARA AUTOMATIZAR LA COMPILACIÓN

- [compilar.py](#)
- [launch gestion incidencias.py](#)

3.3 DISTRIBUIBLE

3.4 EJECUTABLES EN OTROS SISTEMAS OPERATIVOS

- [Mac](#)
 - [Archivo requirements.txt](#)
- [Linux](#)

ANEXO - [Análisis en detalle de los archivos de código de interfaz gráfica de la Versión 2](#)

A-1. [gestion incidencias.py](#)

A-2. [style.css](#)

1. OBJETIVO

Versión 2 de la aplicación:

- Partiendo de la ***app de gestión de incidencias*** creada en base a la **Sesión Experiencial** de la asignatura “***Entornos de Desarrollo***” de 1º de DAM online del curso 2025-2026, se procede a implementar una ***interfaz gráfica para la app*** que permita la interacción del usuario con la infraestructura de capas ya creadas y el sistema de base de datos embebido en SQLite.
- Así mismo, se procede a implementar una **nueva funcionalidad** no disponible en la versión anterior: la posibilidad de dar de ***baja técnicos (pasar a estado no activo)*** con el pertinente cambio de estado de los tickets/incidencias que tuvieran asignados (pasando su estado a abierto/OPEN).
- Creados los nuevos archivos necesarios y con las modificaciones pertinentes en los ya existentes, se procede a un documentado más exhaustivo de los archivos (comentarios en el código).

Versión 3 de la aplicación:

- Por último, se procede a la ***creación de un ejecutable (.exe)*** para su distribución en **Windows** y se comenta el proceso a seguir para la creación de ejecutables en entornos **macOS** y **Linux**.

2. VERSIÓN 2 – APP CON INTERFAZ GRÁFICA

2.1 CÓDIGO

> Consideraciones previas

Puesto que el entorno virtual (.venv) del proyecto nace completamente limpio, sin ninguna librería externa, es necesario **instalar Streamlit** en el proyecto. Si no está instalado hay que ejecutar el siguiente comando en la terminal de PyCharm:

```
pip install streamlit
```

El código de la interfaz se implementa en un nuevo archivo llamado **“gestión_incidentes.py”** en la carpeta del archivo; para no modificar la carpeta del proyecto original se abre un nuevo proyecto en una carpeta **“IncidenciaTickets_V2”** con una copia del original; en esta carpeta se crea el nuevo archivo **“gestión_incidentes.py”**.

Además, se instala **streamlit-option-menu**:

- Para instalarlo se ejecuta el siguiente comando en la terminal de PyCharm:

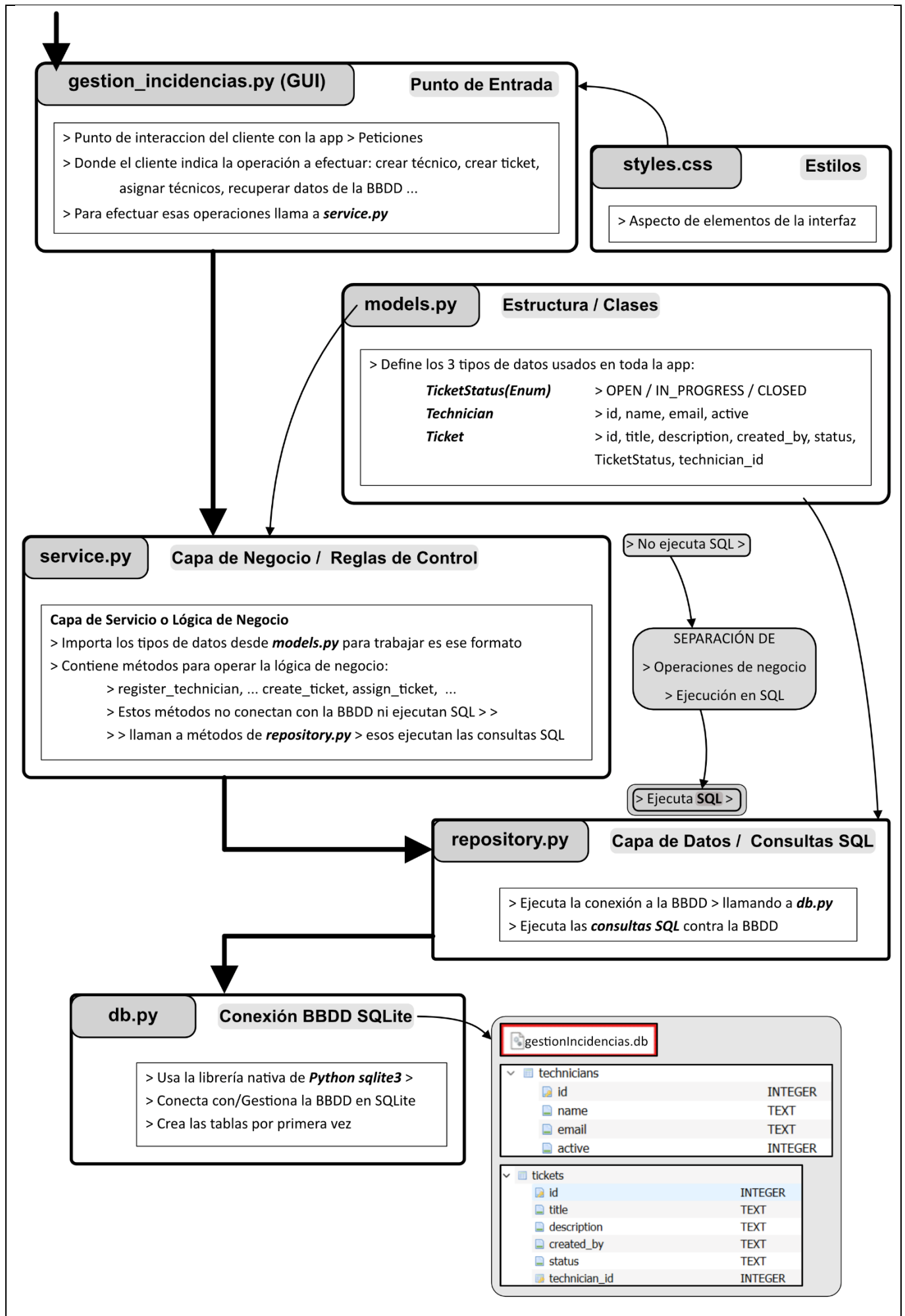
```
pip install streamlit-option-menu
```

- Se trata de un componente **Streamlit** sencillo que permite a los usuarios seleccionar un único elemento de una lista de opciones en un menú.
- Se importa esta dependencia para el diseño de las **pestañas** de la app en lugar de emplear las opciones nativas de Streamlit como **st.tabs** o **st.sidebar.radio**, permitiendo mayor control del mismo:
 - Streamlit es muy bueno para prototipar rápido, pero es muy rígido a la hora de cambiar el diseño visual.
 - **Control de estilos:**
 - Con el uso de **st.tabs** el diseño era básico y gris.
 - El componente **option_menu** permite inyectar un **diccionario de configuración** donde se definen exactamente los colores, bordes y comportamientos del menú. Gracias a esto se fuerzan los colores de las pestañas, se permite diferenciar el aspecto de la pestaña seleccionada frente a las demás.
 - **Integración directa de iconos:**
 - Permite, además, pasar una lista directa de iconos (**icons=["card-list", "person-badge", "gear"]**) que se renderizan automáticamente y con un aspecto muy profesional al lado del texto de cada sección. Con pestañas nativas, sin embargo, se requiere inyectar emojis de texto que no siempre se ven iguales en todos los navegadores o sistemas operativos.

> Esquema básico de la nueva versión app / mapa de interconexión de archivos

La estructura de capas e interconexión de archivos es la misma que en la versión anterior, pudiendo representarse como una cadena de dependencias que va desde la parte más externa (la interacción con el usuario) hasta la más interna (la base de datos). Si bien, en este caso se añade una **nueva capa** en la que se implementan los **archivos que configuran la interfaz**, conforme al siguiente esquema:

- El archivo **“gestion_incidentes.py”** viene a sustituir en su funcionalidad el archivo **“app.py”**, integrando:
 - La **interacción gráfica** con usuario, en lugar de la interacción más limitada mediante terminal como ocurría con **“app.py”**.
 - La llamada al **Servicio** (**“service.py”**) para realizar las operaciones sobre la base de datos y devolver la salida mostrándola en pantalla a través de la interfaz.
- El archivo **“styles.css”** empleado para modificar el **diseño** de base, más rígido y limitado, que proporciona **Streamlit** para la interfaz de la aplicación, aportando un aspecto más profesional y enriquecido.



> Código de la interfaz

En primer lugar y a continuación, se muestra el código de los archivos *“gestion_incidentes.py”* y *“styles.css”* que configuran los elementos de la interfaz gráfica y su diseño respectivamente.

En el caso de *“gestion_incidentes.py”* se realiza una mejora importante respecto a *“app.py”* con respecto a la **inicialización de la base de datos**:

- En la **versión sin interfaz gráfica**, la creación de la base de datos se gestionaba mediante el uso de *import os* y el *if os.path.exists(...)* que comprobaba la existencia en disco duro del archivo *claseExperiencial.db*. En caso de existir ya en disco duro se lanzaba un mensaje pidiendo borrarla, mientras que de no existir se llamaba a la función *init_db()* de *“db.py”* para generar la estructura.
- En la **versión con interfaz gráfica** se delega la **gestión** directamente al **motor de SQLite** y se modifica el archivo *“db.py”* para trabajar en conjunto de manera coherente. Así:
 - Si la base de datos no existe **SQLite la crea**; en caso de existir queda igual sin que se lance ningún error.
 - En cuanto a las tablas, en las sentencias de SQL que las crean dentro de *“db.py”* mediante **CREATE TABLE**, se añade el término **IF NOT EXISTS**. De este modo, si las tablas ya existen no dará error puesto que no intenta crearlas de nuevo.
 - El archivo que contiene la base de datos pasa a llamarse *“gestionIncidentes.db”*.

En cuanto a *“styles.css”* destaca el empleo de la orden *!important*:

- Streamlit está diseñado para que los programadores de Python no tengan que saber nada de diseño web. Por eso, cuando se arranca la app, **Streamlit inyecta automáticamente cientos de reglas CSS de fábrica** (sus propios colores, tamaños y márgenes) directamente en el navegador del usuario final.
- El problema es que estas reglas por defecto se cargan en el último milisegundo y de una forma muy "agresiva". Aunque se escriba en el archivo simplemente *“background-color: #f1f5f9;”* y el navegador lo lea, milisegundos después el código interno de Streamlit lo sobrescribirá.
- La solución es el uso de *!important*. Al añadir *!important* antes del punto y coma final de cada regla que se pretenda que prevalezca, se rompe la jerarquía; indica al navegador que no importa lo que Streamlit cargue después y cómo esté programado el componente original. Esta regla manda sobre todas las demás de forma absoluta.

Además, cabría destacar que no fue posible gestionar el diseño de todos los elementos a través de código incluido en el CSS. En particular, para el componente *streamlit_option_menu*, empleado para crear las **pestañas** (una **barra de navegación**) que separan los servicios de la aplicación (tickets/incidentes, técnicos y gestión), fue necesario crear un **diccionario de estilos** en el código de *“gestion_incidentes.py”*.

Esto se debe a que *streamlit_option_menu* no fue creado por los desarrolladores de Streamlit, sino por un miembro de la comunidad.

Por motivos estrictos de seguridad, cuando se usa un componente que no es oficial, Streamlit no lo mezcla con el resto del código web. En su lugar, crea un *iframe*, que es literalmente, una página web incrustada dentro de otra página web.

El archivo *“style.css”* se carga en la página web principal (la aplicación "padre"). Sin embargo, las reglas de diseño web dictan que **el CSS de una página web principal no puede atravesar la barrera de un iframe** para modificar lo que hay dentro.

Si se intenta cambiar el color del menú usando el *“style.css”*, el navegador choca contra el muro del *iframe* y el menú sigue viéndose con sus colores por defecto, ignorando las órdenes.

La **solución** aportada por el creador de *streamlit_option_menu* fue crear una "puerta trasera" para el diseño: añadió el parámetro *styles={...}* a su código Python, de modo que Streamlit lee ese diccionario antes de construir el *iframe*. El componente coge esos datos, los traduce a lenguaje CSS internamente y los inyecta por dentro del *iframe* en el mismo momento en que lo está construyendo.

Es la única forma técnicamente posible de darle diseño personalizado a esa barra de navegación y hacer que se fusione visualmente con el resto del gestor de incidencias

> Interfaz – Archivo .py:

gestion_incidencias.py

```

# =====
# 0. IMPORTACIONES
# =====
import streamlit as st
from streamlit_option_menu import option_menu # Importación de librería para opciones de menú
# En este caso se emplea para el diseño de pestañas
from service import Service
from db import init_db

# =====
# 1. CONFIGURACIÓN E INICIALIZACIÓN
# =====
# Configuración por defecto de la página
st.set_page_config(
    page_title="Gestor de Incidencias",
    page_icon="🔧",
    layout="wide",
    initial_sidebar_state="collapsed"
)

# Inicialización segura de la BBDD; se delega al motor de SQLite
init_db()

# Instanciación del servicio y guardado en el estado de la sesión
if 'service' not in st.session_state:
    st.session_state.service = Service()

service = st.session_state.service

# =====
# 2. VENTANAS MODALES (DIALOGS)
# =====
#Ventana de confirmación de baja de un técnico

# Decorador para definir una función asociada a ventana de diálogo
@st.dialog("⚠️ Confirmar Baja de Técnico")

# Función asociada a esa ventana:
def dialog_confirmar_baja(id_tecnico, nombre_tecnico):
    st.warning(f"¿Estás completamente seguro de que deseas desactivar a **{nombre_tecnico}**?")
    st.write("Todas sus incidencias en progreso serán desasignadas y volverán a la cola en estado OPEN.")

    col_yes, col_no = st.columns(2)
    with col_yes:
        # Se presiona SI
        if st.button("SI", type="primary", width="stretch"):
            try:
                # Llamada al método para cambiar el estado de un técnico a desactivado
                # Actualizando la BBDD
                st.session_state.service.deactivate_technician(id_tecnico)
                st.rerun() # Re-ejecución de todo el script de Python
            except ValueError as e:
                # Captura de errores desde service.py
                st.error(str(e))
    with col_no:
        # Se presiona CANCELAR
        if st.button("NO, cancelar", width="stretch"):
            st.rerun() # Re-ejecución de todo el script de Python

# =====

```

3. CARGA DEL ARCHIVO CSS PERSONALIZADO

=====

Definición de una función para llevar a cabo la carga del archivo

def load_local_css(file_name):

try:

Apertura y solo lectura del archivo; guardado en una variable del texto contenido

with open(file_name, "r", encoding="utf-8") as f:

markdown carga el texto en la página entre las etiquetas <style>

st.markdown(f'<style>{f.read()}</style>', unsafe_allow_html=True)

Manejo de error en caso de no encontrar el archivo

except FileNotFoundError:

st.warning(f"No se encontró el archivo '{file_name}'.")

Ejecución de la función para cargar el archivo (pasado como parámetro)

load_local_css("style.css")

=====

4. CABECERA

=====

Título de la APP, equivalente a un h1

st.title(" 🚚 SISTEMA DE GESTIÓN DE INCIDENCIAS")

Línea divisoria horizontal

st.markdown("<hr>", unsafe_allow_html=True)

=====

5. MENÚ DE NAVEGACIÓN ("PESTAÑAS")

=====

selected = option_menu(

Devuelve el texto de la selección y lo guarda

en la variable selected

Oculta el título del Menú (ya tenemos el st.title)

menu_title=None,

Textos de las pestañas

options=["Ver Incidencias", "Técnicos", "Gestión y Asignación"],

Iconos de la librería Bootstrap Icons

icons=["card-list", "person-badge", "gear"],

Pestaña por defecto al abrir app > 0 = la primera

default_index=0,

Fuerza a mostrar el menú en horizontal (no vertical) = barra de pestañas

orientation="horizontal",

Diccionario de estilos

styles={

Contenedor principal de las pestañas

"container": {

"padding": "0",

"background-color": "#c8d6e5",

"border": "none",

"box-shadow": "none",

"max-width": "100%",

"margin-bottom": "-2px",

"position": "relative",

"z-index": "10"

},

Envoltura de todas las pestañas

"nav": {

"background-color": "#c8d6e5",

"justify-content": "flex-start",

"margin": "0",

"padding": "0"

},


```

# Cada elemento / pestaña de navegación
"nav-item": {
    "flex": "0 0 auto",
    "margin-right": "5px"
},
# Zona interactiva (de clic) real en la pestaña (todas)
"nav-link": {
    "font-size": "1.05rem",
    "font-weight": "600",
    "color": "#475569",
    "background-color": "#e2e8f0",
    "border": "1px solid #64748b",
    "border-bottom": "1px solid #64748b",
    "border-radius": "8px 8px 0 0",
    "padding": "10px 20px"
},
# Zona interactiva (de clic) real en la pestaña (la seleccionada)
"nav-link-selected": {
    "background-color": "#ffffff",
    "color": "#000000",
    "border-bottom": "2px solid #f1f5f9",
    "font-weight": "700"
}
}
)

# =====
# 6. CONTENIDO DE LAS PESTAÑAS
# =====
# Contenedor grande que contiene todas las operaciones/opciones
with st.container():
    # Inyección de un identificador del contenedor para darle estilo en CSS
    st.markdown("<div id='recuadro-principal'></div>", unsafe_allow_html=True)

    # -----
    # PESTAÑA 1: VER INCIDENCIAS
    # -----
    if selected == "Ver Incidencias":
        col1, col2 = st.columns([1.5, 1])

        # Columna de la izquierda: listado de tickets
        with col1:
            st.subheader("Listado de Tickets")

            # Botones de filtrado
            filtro = st.radio("Filtrar por estado:", ["Todos", "Abiertos (OPEN)"], horizontal=True,
                              label_visibility="collapsed")
            # Llamada a service para recuperar los tickets que cumplen el filtro
            tickets = service.get_all_tickets() if filtro == "Todos" else service.get_open_tickets()
            # Si hay tickets
            if tickets:
                # Desempaquetado y creación de diccionario válido para Streamlit
                datos_tickets = [{**t.__dict__, "status": t.status.value} for t in tickets]
                # Pintado de los datos como tabla
                st.dataframe(datos_tickets, width="stretch")
            else:
                st.info("No hay tickets que mostrar.")

```

```

# Columna de la derecha: creación de un nuevo ticket
with col2:
    st.subheader("🔢 Nuevo Ticket")

# Formulario
with st.form("form_nuevo_ticket", clear_on_submit=True):
    titulo = st.text_input("Título de la incidencia")
    desc = st.text_area("Descripción detallada")
    email = st.text_input("Email del solicitante")

# Botón de envío del formulario
submit_ticket = st.form_submit_button("Crear Ticket", type="primary")

# Al clicar el botón de envío
if submit_ticket:
    try:
        # Llamada a service para crear un nuevo ticket
        nuevo_ticket = service.create_ticket(titulo, desc, email)
        st.success(f"Ticket #{nuevo_ticket.id} creado con éxito.")
        st.rerun()
    except ValueError as e:
        # Manejo de error en caso de algún formato de dato no válido
        st.error(str(e))

# -----
# PESTAÑA 2: TÉCNICOS
# -----
elif selected == "Técnicos":
    col1, col2 = st.columns([1.5, 1])

# Columna de la izquierda: listado de técnicos y apartado de bajas
with col1:
    st.subheader("👤 Plantilla de Técnicos")
    # Llamada al service para recuperar todos los técnicos de la BBDD
    tecnicos = service.get_all_technician()

# Si hay técnicos
if tecnicos:
    # Conversión a diccionario válido para Streamlit y muestra en tabla
    st.dataframe([t.__dict__ for t in tecnicos], width="stretch")

# SECCIÓN DE BAJA DE TÉCNICO
st.markdown("#### ❌ Dar de baja a un técnico")
# Nueva lista con "solo técnicos en activo"
tecnicos_activos = [t for t in tecnicos if t.active]

# Si hay técnicos en activo
if tecnicos_activos:
    # Creación de un formulario
    with st.form("form_baja_tecnico"):
        # Selector desplegable
        # Almacena el ID de la selección
        # aunque en pantalla muestra el nombre (gracias a la función lambda)
        tecnico_id_baja = st.selectbox(
            "Selecciona el técnico a desactivar:",
            [t.id for t in tecnicos_activos],
            format_func=lambda x: next(t.name for t in tecnicos_activos if t.id == x)
        )

# Botón de envío de formulario para dar de baja
if st.form_submit_button("Dar de Baja", type="primary"):
    nombre_tecnico = next(t.name for t in tecnicos_activos if t.id == tecnico_id_baja)
    # Se pasa el ID y el nombre del técnico a la ventana modal programada anteriormente
    dialog_confirmar_baja(tecnico_id_baja, nombre_tecnico)

```

```
# Si NO hay técnicos en activo
else:
    st.info("No hay técnicos activos actualmente.")
```

```
# Si NO hay técnicos
else:
    st.info("No hay técnicos registrados.")
```

```
# Columna de la derecha: creación de un nuevo técnico
```

```
with col2:
```

```
    st.subheader("🔧 Nuevo Técnico")
```

```
    # Formulario
```

```
    with st.form("form_nuevo_tecnico", clear_on_submit=True):
```

```
        nombre = st.text_input("Nombre completo")
```

```
        email_tech = st.text_input("Correo electrónico")
```

```
        submit_tech = st.form_submit_button("Registrar Técnico", type="primary")
```

```
# Si se clicca el botón de envío
```

```
if submit_tech:
```

```
    try:
```

```
        # Llamada al service para registrar el nuevo técnico
```

```
        nuevo_tecnico = service.register_technician(nombre, email_tech)
```

```
        st.success(f"Técnico {nuevo_tecnico.name} registrado.")
```

```
        # Reinicio de la app
```

```
        st.rerun()
```

```
    # Manejo de errores o excepciones (reglas de negocio)
```

```
    except Exception as e:
```

```
        st.error(f"Error: {e}")
```

```
# -----
```

```
# PESTAÑA 3: GESTIÓN Y ASIGNACIÓN
```

```
# -----
```

```
elif selected == "Gestión y Asignación":
```

```
    st.subheader("⚙️ Acciones Operativas")
```

```
# Llamada al service para extraer todos los tickets y todos los técnicos de la BBDD
```

```
todos_tickets = service.get_all_tickets()
```

```
todos_tecnicos = service.get_all_technician()
```

```
# Nueva lista con "solo los técnicos en activo"
```

```
tecnicos_activos = [t for t in todos_tecnicos if t.active]
```

```
# Casi de haber tickets y técnicos en la BBDD
```

```
if todos_tickets and tecnicos_activos:
```

```
    # 2 columnas: una para el panel de Asignación y otra para el panel de Cierre de Tickets
```

```
    col_asig, col_cierre = st.columns(2)
```

```
# Panel de Asignación
```

```
with col_asig:
```

```
    # Formulario
```

```
    with st.form("form_asignar_ticket"):
```

```
        st.markdown("#### 📩 Asignar Técnico a Ticket")
```

```
        # Desplegable para seleccionar el ticket a asignar
```

```
        ticket_id_asignar = st.selectbox("Selecciona Ticket:", [t.id for t in todos_tickets])
```

```
        # Desplegable para seleccionar el técnico a asignar
```

```
        # (mediante ID, pero mostrando en pantalla el nombre)
```

```
        tecnico_id_asignar = st.selectbox(
```

```
            "Selecciona Técnico:",
```

```
            [t.id for t in tecnicos_activos],
```

```
            format_func=lambda x: next(t.name for t in tecnicos_activos if t.id == x)
```

```
        )
```

```
# Botón de envío
if st.form_submit_button("Asignar Ticket", type="primary"):
    try:
        # Llamada al service para la asignación
        service.assign_ticket(ticket_id_asignar, tecnico_id_asignar)
        st.success("¡Asignación realizada con éxito!")
        st.rerun() # Reinicio de la app
    except ValueError as e:
        st.warning(str(e))

# Panel de Cierre
with col_cierre:
    # Formulario
    with st.form("form_cerrar_ticket"):
        st.markdown("#### 🛑 Cerrar Ticket")
        # Desplegable para selección de ticket
        ticket_id_cerrar = st.selectbox("Selecciona Ticket a cerrar:", [t.id for t in todos_tickets])

    # Si se clicla el botón
    if st.form_submit_button("Cerrar Ticket", type="primary"):
        try:
            # Llamada al service para cerrar el ticket
            service.close_ticket(ticket_id_cerrar)
            st.success("¡Ticket cerrado correctamente!")
            st.rerun() # Reinicio de la app
        except ValueError as e:
            st.error(str(e))
    else:
        st.warning("Debe haber al menos un ticket y un técnico registrados para realizar asignaciones.")
```

> Interfaz – Archivo .css:

styles.css

```

/* !important: da prioridad a la orden en la jerarquía CSS para
evitar que Streamlit las sobreescriba con las suyas*/

/* =====
1. FONDO GENERAL Y LIMPIEZA
===== */

/* #root: contenedor principal
.stApp: caja gigante donde Streamlit dibuja la app */
html body div#root div.stApp {
  background-color: #c8d6e5 !important;
}

/* Ocultar la barra superior (Deploy, menú) */

/* header[data-testid="stHeader"]: apunta a la barra superior que Streamlit genera por defecto.*/
html body div#root div.stApp header[data-testid="stHeader"] {
  display: none !important;
  height: 0 !important;
  padding: 0 !important;
  margin: 0 !important;
}

/* =====
2. CONTENEDOR CENTRAL
===== */

/* El contenedor central toma un nombre u otro según la versión:
  block-container
  stMainBlockContainer */
html body div#root div.stApp div.main div.block-container,
html body div#root div.stApp div[data-testid="stMainBlockContainer"] {
  background-color: transparent !important;
  box-shadow: none !important;
  border: none !important;
  padding: 1.5rem 3rem 3rem 3rem !important;
  margin-top: -0.5rem !important; /* Se empuja arriba para llenar el hueco dejado */
  max-width: none !important;
}

/* =====
3. TÍTULO EN RECUADRO INDEPENDIENTE
===== */

/* En Python con Streamlit, la función st.title genera una etiqueta <h1>
Aquí se apunta a esa etiqueta <h1> */
html body div#root div.stApp h1 {
  color: #0f172a !important;
  font-weight: 700 !important;
  background-color: #f1f5f9 !important;
  border: 1px solid #64748b !important;
  padding: 1.5rem 2rem !important;
  border-radius: 8px !important;
  box-shadow: 0 4px 6px -1px rgba(0,0,0,0.05) !important;
  margin-bottom: 0.5rem !important;
  margin-top: 0.5rem !important;
}

/* Líneas separadoras
Busca todas las líneas separadoras dentro de la app (.stApp) */
html body div#root div.stApp hr {
  border: none !important;
  height: 3px !important;
  background-color: #64748b !important;
  margin-top: 1.5rem !important;
  margin-bottom: 2.5rem !important;
}

```

```

/* =====
4. RECUADRO GIGANTE DE CONTENIDO (La "Carpeta")
===== */

/* 1. Limpieza de cualquier margen residual del ancla invisible
element-container es el contenedor tipo que Streamlit pone a cada elemento
has() busca dentro del elemento el que tiene el marcador indicado #recuadro-principal */
div.element-container:has(#recuadro-principal) {
  display: none !important;
  margin: 0 !important;
  padding: 0 !important;
}

/* 2. Tirón del recuadro hacia arriba para cerrar el hueco
Busca el bloque vertical que contienen ese elemento contenedor a mover arriba */
div[data-testid="stVerticalBlock"]:has(> div.element-container #recuadro-principal) {
  background-color: #f1f5f9 !important;
  border: 1px solid #64748b !important;
  border-radius: 0 8px 8px 8px !important;
  padding: 2.5rem !important;
  box-shadow: 0 4px 6px -1px rgba(0,0,0,0.05) !important;
  position: relative !important;
  z-index: 1 !important;

  /* ¡MODIFICAR los px para mover la caja a la altura estéticamente adecuada! */
  margin-top: -38px !important;
}

/* =====
5. INPUTS, BOTONES Y FORMULARIOS (Internos)
===== */

/* Selecciona todos los formularios */
html body div#root div.stApp div[data-testid="stForm"] {
  border: 1px solid #cbd5e1 !important;
  border-radius: 10px !important;
  padding: 1.5rem !important;
  background-color: #e2e8f0 !important;
  box-shadow: 0 2px 4px rgba(0,0,0,0.05) !important;
}

/* Selecciona todas las cajas de texto */
html body div#root div.stApp .stTextInput input,
html body div#root div.stApp .stTextArea textarea,
html body div#root div.stApp .stSelectbox div[data-baseweb="select"] {
  border-radius: 6px !important;
  border: 1px solid #cbd5e1 !important;
  background-color: #ffffff !important;
}

/* Resalta la casilla en que se está escribiendo */
html body div#root div.stApp .stTextInput input:focus,
html body div#root div.stApp .stTextArea textarea:focus {
  border-color: #3b82f6 !important;
  box-shadow: 0 0 1px #3b82f6 !important;
}

/* Botones secundarios
div.stButton > button busca la caja que contiene el botón
y le aplica el formato al botón en si, que está dentro de la caja */
html body div#root div.stApp div.stButton > button {
  background-color: #e2e8f0 !important;
  color: #0f172a !important;
  border: 1px solid #94a3b8 !important;
  border-radius: 6px !important;
  transition: all 0.2s ease !important;
}

html body div#root div.stApp div.stButton > button:hover {
  background-color: #cbd5e1 !important;
  border-color: #64748b !important;
}

```

```
/* Botón primario (Formularios)
  Aplica a los botones del tipo formulario
  y a los marcados en el código Python como type="primary" */
html body div#root div.stApp div.stFormSubmitButton > button,
html body div#root div.stApp button[kind="primary"] {
  background-color: #3b82f6 !important;
  color: white !important;
  border: none !important;
  border-radius: 6px !important;
  font-weight: 500 !important;
}

html body div#root div.stApp div.stFormSubmitButton > button:hover,
html body div#root div.stApp button[kind="primary"]:hover {
  background-color: #2563eb !important;
  color: white !important;
}
```

> **Modificaciones en el backend:**> **Archivo gestor de la BBDD:** **db.py**

- Se modifica el nombre del archivo de la base de datos que generará la aplicación en el disco local, cambiando de *"claseExperiencial.db"* a *"gestionIncidencias.db"*:

```
DB_NAME = "gestionIncidencias.db"
```

- Se modifica el método de inicialización de la base de datos *def init_db()*, para incluir la condición *IF NOT EXISTS* en las sentencias de creación de las tablas:

```
# Inicialización de la BBDD y tablas
def init_db() -> None:
    ...
    cursor.execute("""
    CREATE TABLE IF NOT EXISTS technicians (
        ...
    )
    """)

    cursor.execute("""
    CREATE TABLE IF NOT EXISTS tickets (
        ...
```

> **Repositorio (operaciones SQL):** **repository.py**

- Dentro de la clase *TechnicianRepository*, asociada a las operaciones relativas a la tabla de técnicos (tabla technicians), se añade un *nuevo método deactivate()* para desactivar un técnico, cambiando su estado de activo a 0 en lugar de 1:

```
# Desactiva un técnico > SQL: UPDATE technicians SET active = 0 WHERE id = ?
def deactivate(self, technician_id: int) -> bool:
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute(
        "UPDATE technicians SET active = 0 WHERE id = ?",
        (technician_id,)
    )
    conn.commit()
    updated = cursor.rowcount > 0
    conn.close()
    # Devuelve un booleano informando de operación o no exitosa
    return updated
```

- Dentro de la clase *TicketRepository*, asociada a las operaciones relativas a la tabla de tickets (tabla tickets), se añade un *nuevo método unassign_active_tickets_from_technician()* para liberar los tickets asociados a un técnico que pase a estado inactivo, cambiando el estado de esos tickets a "OPEN":

```
# NUEVO MÉTODO: Liberar tickets en progreso al dar de baja a un técnico dado su ID
# > SQL: UPDATE tickets SET status = ?, technician_id = NULL WHERE technician_id = ? AND status = ?
def unassign_active_tickets_from_technician(self, technician_id: int) -> int:
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute(
        "UPDATE tickets SET status = ?, technician_id = NULL WHERE technician_id = ? AND status = ?",
        (TicketStatus.OPEN.value, technician_id, TicketStatus.IN_PROGRESS.value)
    )
    conn.commit()
    rows_affected = cursor.rowcount
    conn.close()
    # Devuelve el número de filas afectadas
    return rows_affected
```


> Servicio: **service.py**

- Se añade un nuevo método para llevar a cabo la desactivación de un técnico de la base de datos. El método implica:
 - La llamada al método **find_by_id()** de **TechnicianRepository** que busca al técnico por ID de la clase **TechnicianRepository**
 - La llamada al método **unassign_active_tickets_from_technician()** de **TicketRepository** que pone a "OPEN" el status de los tickets asociados al técnico que pasa a estar desactivado.
 - La llamada al método **deactivate()** de **TechnicianRepository** para poner el estado del técnico a 0 (desactivado).

```
# Desactivación estricta de técnicos
def deactivate_technician(self, technician_id: int) -> None:
    # Busca el técnico por ID
    technician = self.technician_repository.find_by_id(technician_id)
    if technician is None:
        raise ValueError(f"El técnico con ID {technician_id} no existe.")

    if not technician.active:
        raise ValueError(f"El técnico {technician.name} ya se encuentra inactivo.")

    # 1. Se liberan sus incidencias en progreso (vuelven a OPEN)
    self.ticket_repository.unassign_active_tickets_from_technician(technician_id)

    # 2. Se desactiva al técnico
    update = self.technician_repository.deactivate(technician_id)
    if not update:
        raise ValueError("No se pudo desactivar el técnico debido a un error en la base de datos.")
```

2.2 EJECUCIÓN EN TERMINAL DE LA APLICACIÓN CON INTERFAZ GRÁFICA**> Puesta en funcionamiento de la app**

Seguidamente y puesto que **Streamlit** levanta un servidor web local, **no se ejecuta con el comando python normal**. En la terminal de PyCharm, una vez ubicado en la carpeta del proyecto hay que ejecutar este comando:

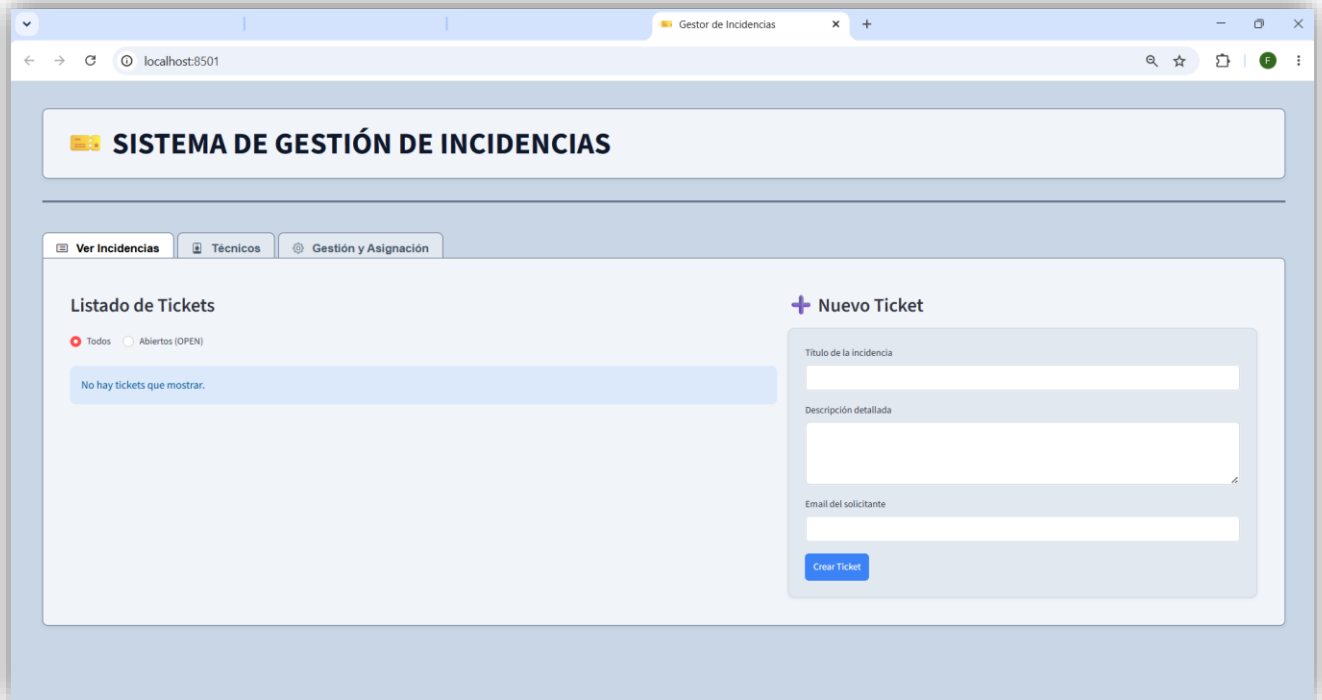
```
streamlit run gestion_incidencias.py
```

Al ejecutarlo:

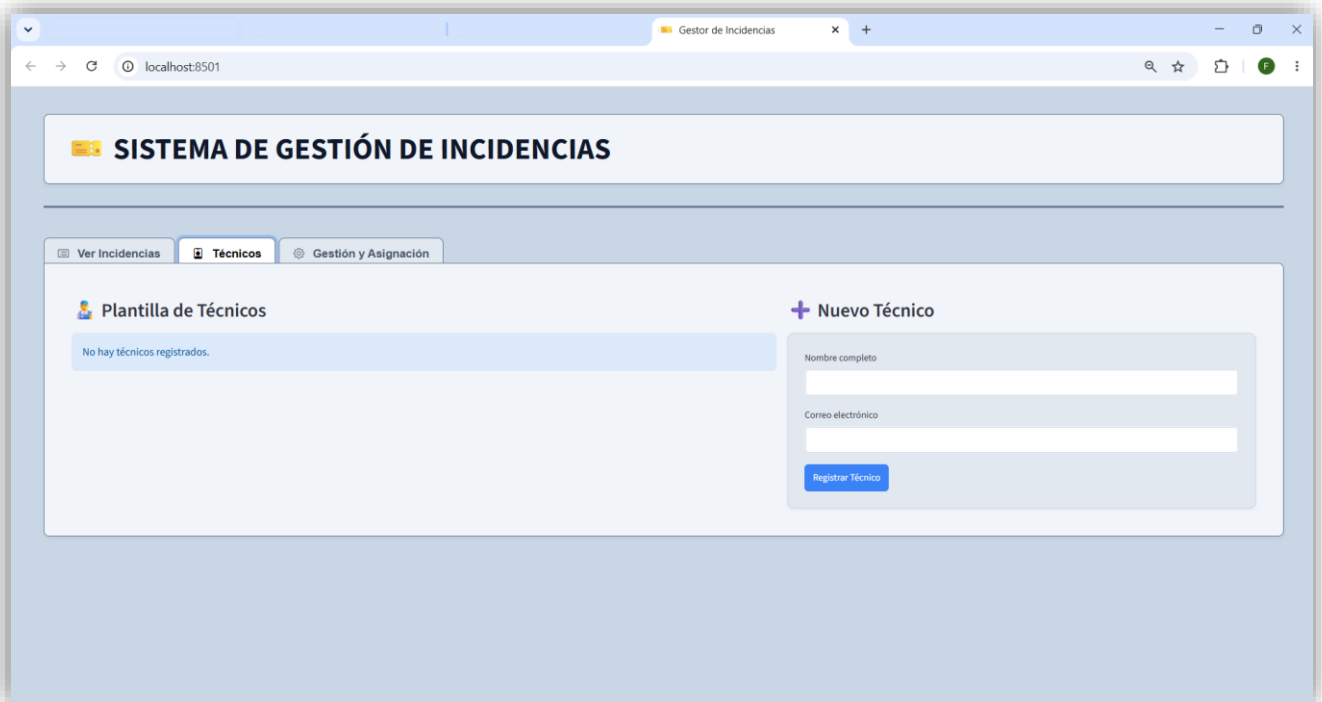
- Se abre automáticamente el navegador web (generalmente suele ser en **http://localhost:8501**).
- Aparece la interfaz completamente funcional, permitiendo interactuar con pestañas, formularios y botones.

> **Aspecto de la interfaz y pruebas funcionales**> **Aspecto inicial con la base de datos aún no creada**

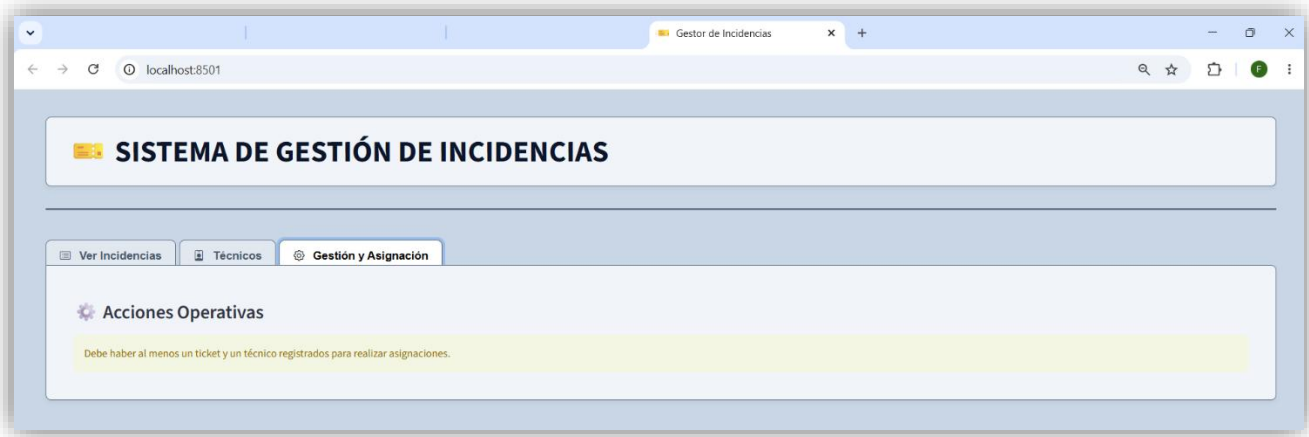
Al arrancar la aplicación por primera vez, aún no existe el archivo de base de datos; la aplicación crea el archivo ***“gestionIncidentes.db”*** con las correspondientes tablas, pero éstas aún no contienen registros. Por tanto todos los campos de cada una de las pestañas se muestran sin datos.



Pestaña de Incidencias



Pestaña de Técnicos



Pestaña de Gestión y Asignación

> Proceso de creación de tickets

En la pestaña “**Ver Incidencias**”, rellenando el formulario de “**Nuevo Ticket**” con valores válidos y clicando “**Crear Ticket**”, se graban en la base de datos y se pasan a mostrar en el listado de la parte izquierda.

A screenshot of the 'Nuevo Ticket' form. It has a title '+ Nuevo Ticket'. The form contains three input fields: 'Título de la incidencia' with the value 'Incidencia 1', 'Descripción detallada' with the value 'Rotura de pantalla', and 'Email del solicitante' with the value 'trabajador1@empresa.com'. Below the email field is a button labeled 'Crear Ticket'.

Relleno de formulario

A screenshot of the 'Ver Incidencias' tab in the application. It shows a section titled 'Listado de Tickets' with two radio buttons: 'Todos' (selected) and 'Abiertos (OPEN)'. Below this is a table with the following data:

id	title	description	created_by	status	technician_id
1	Incidencia 1	Rotura de pantalla	trabajador1@empresa.com	OPEN	None
2	Incidencia 2	No arranca el PC	secretario@empresa.com	OPEN	None

Tickets ya introducidos

Tabla: tickets

	id	title	description	created_by	status	technician_id
	Fil...	Filtro	Filtro	Filtro	Filtro	Filtro
1	1	Incidencia 1	Rotura de pantalla	trabajador1@empresa.com	OPEN	NULL
2	2	Incidencia 2	No arranca el PC	secretario@empresa.com	OPEN	NULL

Tabla tickets de la base de datos actualizada y datos persistidos

> Proceso de creación de técnicos

En la pestaña “**Técnicos**”, rellenando el formulario de “**Nuevo Técnico**” con valores válidos y clicando “**Registrar Técnico**”, se graba en la base de datos y se pasan a mostrar en el listado de la parte izquierda.

+ Nuevo Técnico

Nombre completo

Técnico 1 Ejemplo

Correo electrónico

correotecnico1@empresa.com

Registrar Técnico

Relleno de formulario

Ver Incidencias | **Técnicos** | Gestión y Asignación

Plantilla de Técnicos

id	name	email	active
1	Técnico 1 Ejemplo	correotecnico1@empresa.com	☒
2	Técnico 2 ejemplo	correotecnico2@empresa.com	☒

✗ Dar de baja a un técnico

Selecciona el técnico a desactivar:

Técnico 1 Ejemplo

Dar de Baja

Técnicos ya introducidos; habilita la opción de darlos de baja

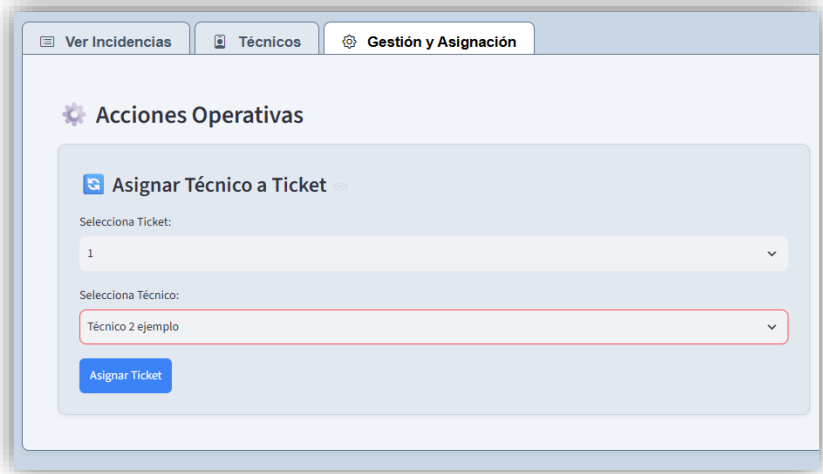
Tabla: technicians

	id	name	email	active
	Fil...	Filtro	Filtro	Filtro
1	1	Técnico 1 Ejemplo	correotecnico1@empresa.com	1
2	2	Técnico 2 ejemplo	correotecnico2@empresa.com	1

Tabla technicians de la base de datos actualizada y datos persistidos

> Asignación de una incidencia a un técnico

En la pestaña “**Gestión y Asignación**”, seleccionando ticket y técnico y clicando “**Asignar Ticket**” se realiza la asignación correspondiente, que puede comprobarse posteriormente en la pestaña “Ver incidencias”. Por ejemplo, asignando el técnico 2 a la incidencia 1:



Ver Incidencias | Técnicos | **Gestión y Asignación**

Acciones Operativas

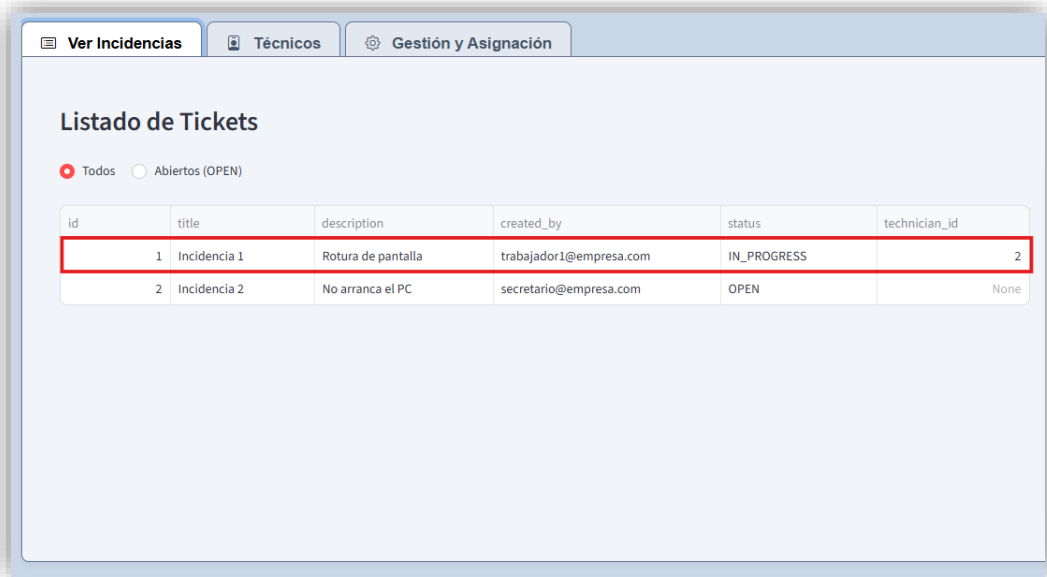
Asignar Técnico a Ticket

Selecciona Ticket: 1

Selecciona Técnico: Técnico 2 ejemplo

Asignar Ticket

Selección de ticket 1 y técnico 2 para asignación



Ver Incidencias | Técnicos | Gestión y Asignación

Listado de Tickets

Todos Abiertos (OPEN)

id	title	description	created_by	status	technician_id
1	Incidencia 1	Rotura de pantalla	trabajador1@empresa.com	IN_PROGRESS	2
2	Incidencia 2	No arranca el PC	secretario@empresa.com	OPEN	None

Comprobación de la asignación realizada correctamente

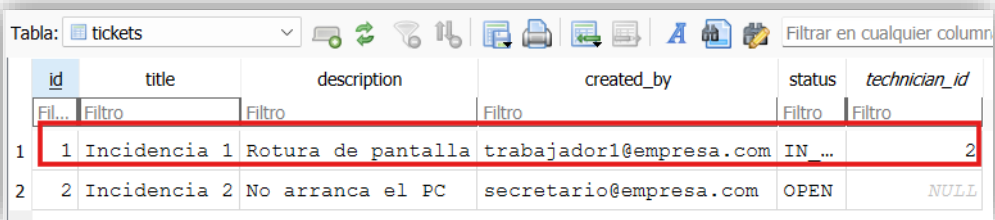


Tabla: tickets

	id	title	description	created_by	status	technician_id
1	1	Incidencia 1	Rotura de pantalla	trabajador1@empresa.com	IN ...	2
2	2	Incidencia 2	No arranca el PC	secretario@empresa.com	OPEN	NULL

Tabla tickets de la base de datos actualizada y datos persistentes

Asignado el ticket 2 al técnico 1 queda:

Ver Incidencias

Técnicos

Gestión y Asignación

Listado de Tickets

Todos

Abiertos (OPEN)

id	title	description	created_by	status	technician_id
1	Incidencia 1	Rotura de pantalla	trabajador1@empresa.com	IN_PROGRESS	2
2	Incidencia 2	No arranca el PC	secretario@empresa.com	IN_PROGRESS	1

Comprobación de la asignación realizada correctamente

> Cierre de una incidencia

En la pestaña “Gestión y Asignación”, seleccionando el ticket en el formulario de la derecha y clicando “Cerrar Ticket” se cambia el estado del ticket seleccionado (si está abierto) a cerrado, como puedo comprobarse para el ticket 2 en las siguientes capturas de pantalla:

Cerrar Ticket

Selecciona Ticket a cerrar:

2

Cerrar Ticket

Selección de ticket 2 para cerrar la incidencia asociada

Ver Incidencias

Técnicos

Gestión y Asignación

Listado de Tickets

Todos

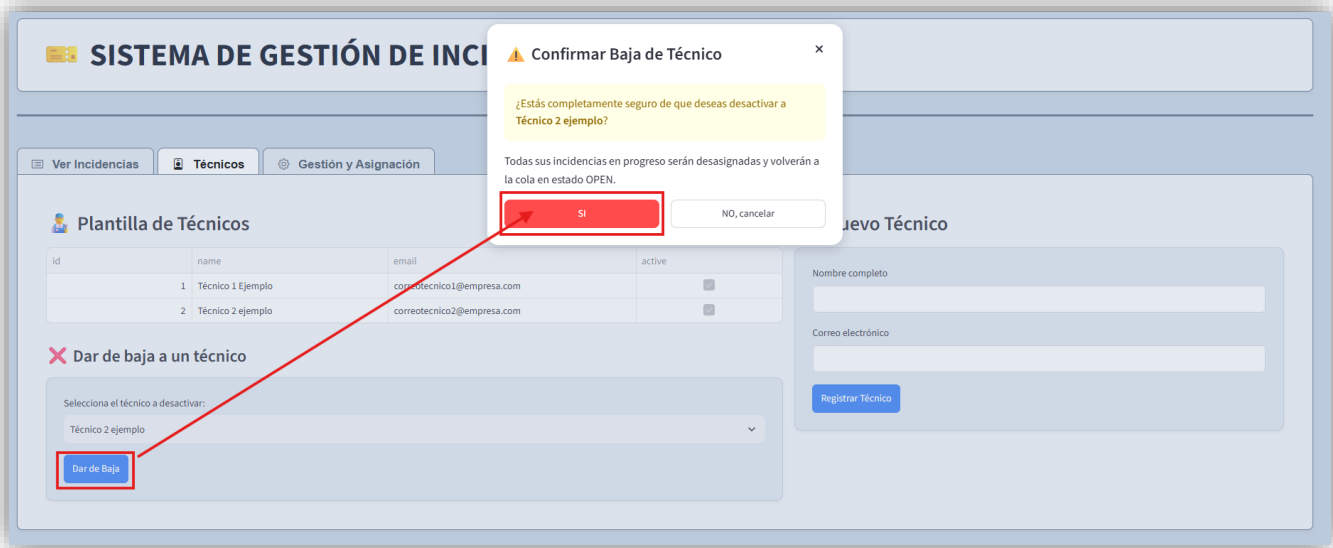
Abiertos (OPEN)

id	title	description	created_by	status	technician_id
1	Incidencia 1	Rotura de pantalla	trabajador1@empresa.com	IN_PROGRESS	2
2	Incidencia 2	No arranca el PC	secretario@empresa.com	CLOSED	1

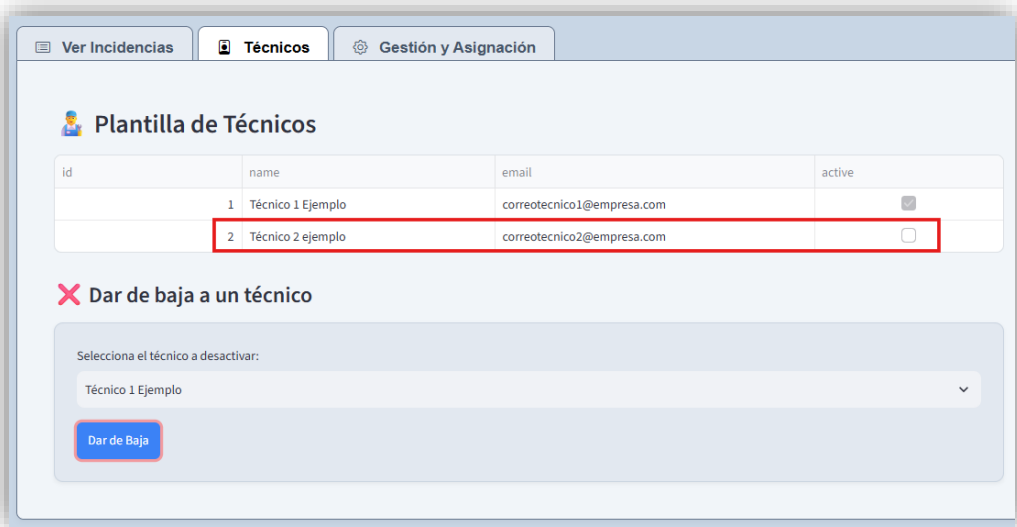
Comprobación de ejecución correcta de la operación de cierre

> Desactivación de un técnico

En la pestaña “**Técnicos**”, se puede desactivar un técnico que se encuentre en estado activo seleccionándolo desde el desplegable del apartado inferior y clicando “**Dar de Baja**”. En las siguientes capturas puede apreciarse el procedimiento para el técnico 2, observando en la pestaña “**Técnicos**” como queda desactivado y a su vez, en la pestaña “**Ver Incidencias**” como al tener asignado un ticket “**en progreso**”, ese ticket pasa a quedar desasignado y en estado de “**open**”:



Confirmación de baja del técnico designado



Se muestra el estado como no activo



Los tickets asociados al técnico son devueltos al estado de OPEN

> Gestión de Reglas de Negocio no permitidas

En este apartado se presenta la comprobación del funcionamiento adecuado de la app en respuesta a:

- **La introducción de datos no válidos a través de los formularios:**

> Pestaña “**Ver Incidencias**”:

- El **título** del ticket no puede estar **vacío**

The screenshot shows a web form titled '+ Nuevo Ticket'. It has three input fields: 'Título de la incidencia' (empty), 'Descripción detallada' (containing 'Ticket inválido por título'), and 'Email del solicitante' (containing 'ejemplo@email.es'). A blue 'Crear Ticket' button is at the bottom. A red error message is displayed below the form: 'ATENCIÓN: El título del ticket no puede estar vacío.'

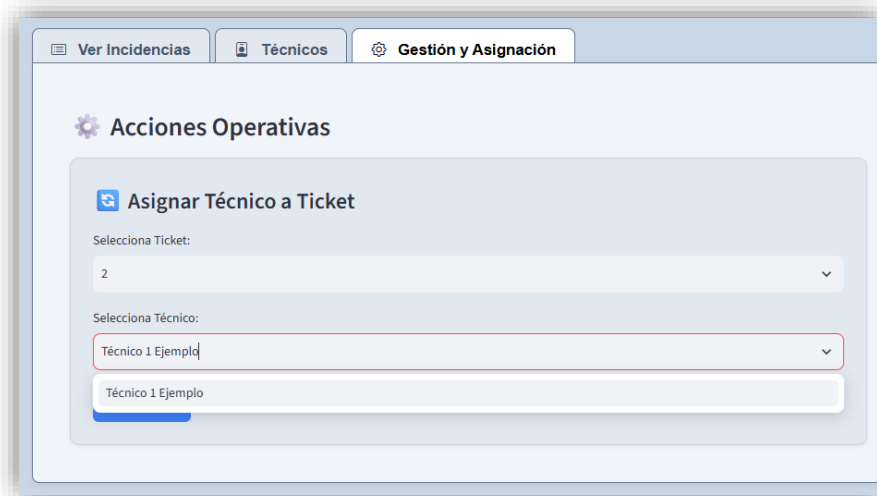
- El **email** del creador de un ticket debe ser válido (**contener @**)

The screenshot shows the same web form. The 'Título de la incidencia' field now contains 'Prueba no válido'. The 'Descripción detallada' field contains 'Ticket no válido por email sin @'. The 'Email del solicitante' field contains 'fran.es'. The blue 'Crear Ticket' button is still present. A red error message is displayed below the form: 'ATENCIÓN: El email del creador debe contener el carácter '@'.'

- Los intentos de acciones no permitidas por las Regla de Negocio:

> Pestaña “Gestión y Asignación”:

- No se puede asignar un **ticket** a un **técnico inactivo**



Ver Incidencias | Técnicos | Gestión y Asignación

Acciones Operativas

Asignar Técnico a Ticket

Selecciona Ticket:

2

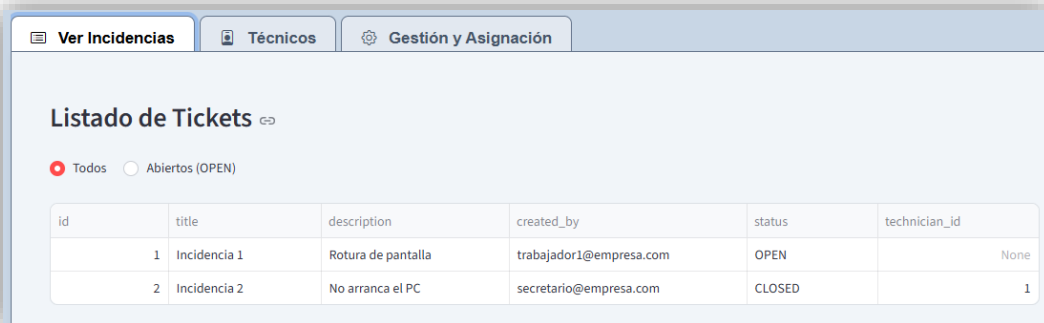
Selecciona Técnico:

Técnico 1 Ejemplq

Técnico 1 Ejemplo

Solo son seleccionables los técnicos activos

- No se puede cerrar un **ticket ya cerrado**



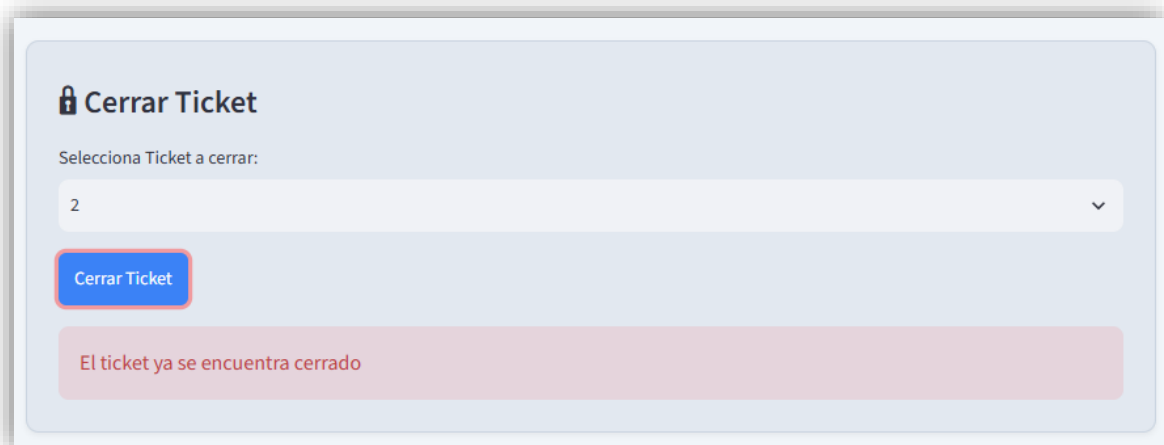
Ver Incidencias | Técnicos | Gestión y Asignación

Listado de Tickets

☒ Todos ☐ Abiertos (OPEN)

id	title	description	created_by	status	technician_id
1	Incidencia 1	Rotura de pantalla	trabajador1@empresa.com	OPEN	None
2	Incidencia 2	No arranca el PC	secretario@empresa.com	CLOSED	1

El ticket 2 está ya cerrado



Cerrar Ticket

Selecciona Ticket a cerrar:

2

Cerrar Ticket

El ticket ya se encuentra cerrado

La acción de cierre es imposible

- No se puede asignar un **ticket sin técnico asignado**

Ver Incidencias

Técnicos

Gestión y Asignación

Listado de Tickets

Todos

Abiertos (OPEN)

id	title	description	created_by	status	technician_id
1	Incidencia 1	Rotura de pantalla	trabajador1@empresa.com	OPEN	None
2	Incidencia 2	No arranca el PC	secretario@empresa.com	CLOSED	1
3	Incidencia 3	Prueba intento cerrar ticket ya cerrado	fran@hot.es	CLOSED	1
4	Incidencia 4	Prueba cerrar ticket sin técnico asignado	fran@hot.es	OPEN	None

El ticket 4 no tiene técnico asignado

 Cerrar Ticket

Selecciona Ticket a cerrar:

4

▼

Cerrar Ticket

ATENCIÓN: No se puede cerrar un ticket sin un técnico asignado.

La acción de cierre es imposible

3. VERSIÓN 3 – VERSIÓN EJECUTABLE DE LA APP CON INTERFAZ GRÁFICA

3.1 PROCEDIMIENTO DE COMPILACIÓN

> Consideraciones previas

Para trabajar en la construcción de un ejecutable sin modificar los archivos del proyecto contenido en la carpeta "IncidenciaTickets_V2" se hace una copia de esta carpeta bajo la denominación "IncidenciaTickets_V3".

Además, se hace necesario instalar la librería **PyInstaller** para construir un ejecutable **.exe** (para entorno **Windows**), empleando para ello el siguiente comando en la terminal de PyCharm:

```
pip install pyinstaller
```

Para hacer un ejecutable de toda la app es necesario tener en cuenta que **Streamlit no fue diseñado originalmente para ser una aplicación de escritorio tradicional (un .exe al que le haces doble clic), sino para ser una aplicación web**. Sin embargo, como Python es muy flexible, existen métodos para "empaquetarlo" y que funcione como un ejecutable. Cabría nombrar 3 opciones, ordenadas desde la más sencilla a la más profesional:

- **El método "Falso Ejecutable" (Archivo .bat):** si los ordenadores donde se va a usar la app ya tienen Python instalado, no es necesario compilar nada. Se puede crear un archivo de texto, llamarlo **Iniciar_App.bat** (en Windows) y escribir esto dentro:

```
@echo off  
python -m streamlit run gestion_incidentes.py
```

Al hacer doble clic en ese archivo .bat, se abrirá la consola un segundo y lanzará la app directamente en el navegador.
- **El método* "Ejecutable Real" (PyInstaller) – Opción seleccionada:** para generar un **.exe** real que se pueda copiar en un ordenador que no tenga Python instalado, se puede usar la librería PyInstaller. Como **Streamlit** se lanza con un comando especial (**streamlit run...**), no se puede convertir "**gestion_incidentes.py**" directamente. Es necesario crear un archivo "lanzador" que llamamos "**launch_gestion_incidentes.py**".
- **La Alternativa Recomendada, el Despliegue Web:** dado que se ha construido una herramienta con múltiples usuarios (técnicos, base de datos de tickets, etc.), la forma natural de usar **Streamlit** es subirla a un servidor (o a la nube). De este modo no hay ejecutables, nadie tiene que instalar nada; los técnicos simplemente entran a una dirección web (ej: incidencias.tuempresa.com) desde cualquier navegador o incluso desde su teléfono móvil, y todos comparten la misma base de datos "tickets.db" centralizada.

* **Nota:** Este método puede generar ejecutables muy pesados (a veces +100MB) y algunos antivirus de Windows los detectan como "sospechosos" porque empaquetan un servidor web entero.

> Obstáculos e Hitos en el procedimiento de empaquetado

Tal y como se indica se opta por la creación de un **ejecutable (.exe)** distribuible.

Hito 1 - El obstáculo arquitectónico de Streamlit (creación de un archivo lanzador):

Como PyInstaller no puede "abrir una terminal y escribir un comando", se hizo necesario crear un archivo intermedio ("**launch_gestion_incidentes.py**") que hiciera ese trabajo desde dentro del propio código Python.

La necesidad nace de la forma en la que funciona **Streamlit**:

- **Un script de Python normal:** si se construye, por ejemplo, una calculadora básica en Python, y se ejecuta haciendo doble clic o escribiendo **python calculadora.py**, el código se lee de arriba a abajo y no hay problema. **PyInstaller** es capaz de empaquetar esto directamente sin problemas.
- **Un script de Streamlit:** esta **app** no es un código lineal normal. Es un **servidor web interactivo**. Para que arranque, normalmente tienes que abrir una terminal y escribir un comando específico: "**streamlit run gestion_incidentes.py**". **Si se le pidiera a PyInstaller compilar "gestion_incidentes.py" directamente, el .exe resultante leería el código de arriba a abajo en milisegundos, no arrancaría ningún servidor web y se cerraría instantáneamente sin mostrar nada.**

La finalidad del archivo intermedio ("**launch_gestion_incidentes.py**") es importar el **módulo interno de línea de comandos de Streamlit** (**from streamlit.web import cli as stcli**). De esta forma, el **.exe** arranca leyendo "**launch_gestion_incidentes.py**" y este, a su vez, simula que alguien ha escrito el comando en la consola inyectando una serie de argumentos (**sys.argv = ["streamlit", "run", ...]**) y encendiendo el servidor internamente (**sys.exit(stcli.main())**).

En resumen, puesto que PyInstaller no puede arrancar un servidor de Streamlit de forma nativa, se decide crear un archivo "lanzador" que sirviera como puente para encender el motor web internamente simulando los comandos de la terminal. Este archivo se convirtió en el objetivo principal a compilar.

Hito 2 – Primera compilación y error "Not Found":

Se compiló por primera vez el archivo "lanzador" con la siguiente orden en terminal de PyCharm:

```
pyinstaller --onefile --windowed launch_gestion_incidentes.py
```

--onefile: mete todo (Python, Streamlit y tu código) en un solo archivo **.exe**.

--windowed: evita que salga la ventana negra de MS-DOS por detrás.

Al poner en marcha el ejecutable obtenido, el navegador abría una ventana, pero mostraba un error **404 Not Found**. ¿Qué ocurría? **PyInstaller dejaba fuera la carpeta con los archivos internos de diseño y gráficos de Streamlit**. La solución fue añadir el comando de recolección total de Streamlit (**--collect-all streamlit**) para obligar al compilador a llevarse las carpetas ocultas de diseño del propio **Streamlit**.

Hito 3 – Error de dependencias ocultas "PackageNotFoundError":

PyInstaller es muy agresivo intentando que los **.exe** pesen lo menos posible; **empaqueta el código de Streamlit**, pero por defecto **borra los "metadatos"** (como el archivo que guarda el número de versión de la librería) para ahorrar espacio.

Cuando **Streamlit** arranca hace una comprobación interna para leer su propia versión. Al ver que **PyInstaller** le ha quitado esos metadatos, falla devolviendo ese **PackageNotFoundError**.

La solución es una orden explícita (**--copy-metadata streamlit**) a **PyInstaller** en la terminal para que copie esos metadatos de Streamlit en el ejecutable, compilando con:

```
pyinstaller --onefile --windowed --copy-metadata streamlit launch_gestion_incidentes.py
```

Hito 4 – El problema de las rutas ("File does not exist"):

La ejecución mostraba un error del tipo *"Error: Invalid value: File does not exist: C:\Users\...\AppData\Local\Temp\MEIXXXX\gestion_incidentes.py"*.

El problema es que cuando *PyInstaller* crea un ejecutable con *--onefile*, al hacerle doble clic, el sistema descomprime todo el contenido en una **carpeta temporal y oculta de Windows llamada *_MEIPASS***.

El lanzador estaba buscando correctamente ahí el archivo *"gestion_incidentes.py"*, pero el error indicaba que no encontraba el archivo de la aplicación principal ahí. La razón es que el comando de compilación solo mencionaba el archivo lanzador. *PyInstaller* por sí solo **no es capaz de saber que también debía llevarse el archivo *"gestion_incidentes.py"* ni la hoja de estilos *"style.css"***.

La solución lógica a este problema era decirle explícitamente a *PyInstaller* que incluyera estos archivos adicionales. Para ello, la herramienta cuenta con el parámetro *--add-data*, el cual permite **especificar qué archivos extra deben incrustarse dentro del ejecutable** y en qué ruta interna deben colocarse (en este caso, en la raíz del paquete, representada con un punto *.*). Se intentó una compilación añadiendo este parámetro directamente en la consola de comandos:

```
pyinstaller --onefile --windowed --copy-metadata streamlit
--add-data "gestion_incidentes.py;" --add-data "style.css;" launch_gestion_incidentes.py
```

Hito 5 – Problema con la terminal de comandos; creación del script compilador:

A pesar de haber añadido el parámetro *--add-data* en el comando, al ejecutar la app compilada seguía apareciendo exactamente el mismo error: **el archivo no existía dentro de la carpeta *_MEIPASS***.

El **problema** ya no era de *PyInstaller*, sino de la propia **terminal**. Al intentar incluir los archivos usando *--add-data "gestion_incidentes.py;"*, **la terminal de comandos interpretaba mal el carácter especial del punto y coma (;)**, que en muchos entornos de consola se utiliza como un **separador de comandos** diferentes. Como consecuencia, la consola ignoraba esa parte de la orden de forma totalmente silenciosa y volvía a compilar el ejecutable dejándose los archivos fuera una vez más.

Se hizo evidente que seguir escribiendo comandos kilométricos en la terminal era inviable. Para esquivar las limitaciones y errores de lectura de la consola de Windows, se implementó una doble **solución** basada en código:

- **Modificar el archivo lanzador:** se incluyó en él una estructura condicional para que el script supiera detectar de manera automática si se estaba ejecutando como un archivo *.exe*. De ser así, se le ordena que capture la ruta absoluta exacta de la carpeta secreta *sys._MEIPASS* y se le pase a *Streamlit*.
- **Crear un script compilador ("*compilar.py*"):** se trasladaron todas las instrucciones a un archivo de Python puro que llama de forma interna al **motor de *PyInstaller* (*PyInstaller.__main__.run*)**. La creación de este archivo permitió usar el módulo *os* de Python para calcular matemáticamente las rutas absolutas exactas del disco duro (mediante *os.path.abspath*) e inyectarlas de forma segura junto al comando *--add-data*. **Al enviarle el texto directamente desde Python al motor de *PyInstaller***, se hace imposible la mala interpretación del punto y coma, logrando por fin la incrustación física y exitosa de los archivos en el ejecutable.

Hito 6 – Error de dependencias ocultas “ModuleNotFoundError”:

Una vez que el script compilador logró incrustar con éxito el archivo “*gestion_incidencias.py*”, la aplicación por fin pudo leer el código. Sin embargo, al ejecutar el *.exe* devolvía un error de tipo “*ModuleNotFoundError*”.

Ahora el problema era que el programa no encontraba el componente externo *streamlit_option_menu* ni las librerías propias del *backend* (los archivos “*db.py*”, “*models.py*”, “*repository.py*” y “*service.py*”); al usar *--add-data*, PyInstaller coge los archivos (*gestion_incidencias.py*, *service.py*, etc.) y los mete en el *.exe* como si fueran simples archivos de texto, como el *.css*. No los abre. No los lee. Y como no los lee, **no ve los import que hay dentro**. Por tanto, ignora por completo que tu proyecto necesita librerías externas para funcionar.

Es decir, el problema residía en un “punto ciego” del análisis de *PyInstaller*: como el script principal (“*launch_gestion_incidencias.py*”) llamaba al archivo “*gestion_incidencias.py*” pasándole el nombre como una cadena de texto para simular la consola, el compilador no era capaz de leer el interior de ese archivo, por lo que ignoraba por completo qué dependencias necesitaba para funcionar y las dejaba fuera de la compilación; así, al intentar convertir el archivo con PyInstaller, este lo leía de arriba a abajo buscando la palabra mágica: **import**:

- Lee **import sys**: mete la librería del sistema en el paquete.
- Lee **from streamlit...**: mete el motor de *Streamlit* en el paquete
- En un programa normal, al escribir **import gestion_incidencias**, el detective lo habría visto y lo habría metido dentro del *.exe* automáticamente. Pero en el archivo “*launch_gestion_incidencias.py*”, la orden para arrancar la aplicación es “*sys.argv = [“streamlit”, “run”, “gestion_incidencias.py”]*”; para el detective *PyInstaller*, “*gestion_incidencias.py*” es solo un trozo de texto entre comillas. No es lo suficientemente listo para entender que ese texto es en realidad un archivo vital que necesita empaquetar. Como no ve ningún **import**, no lo mete.

La **solución** fue doble:

- Para el **backend**: se colocaron **manualmente** las **importaciones del backend** en la parte superior del archivo lanzador (**import db, import models, import repository, import service**). Aunque el lanzador no usa estos archivos directamente, al importarlos se obliga a *PyInstaller* a rastrearlos y empaquetar toda la lógica y las librerías (como *SQLite3*) de forma automática.
- Para el **componente de terceros**: se aprovechó el nuevo archivo “*compilar.py*” para integrar el comando *--collect-all 'streamlit_option_menu'* de forma limpia y segura dentro de la lista de argumentos. Esto obligó a *PyInstaller* a empaquetar no solo el código Python del menú, sino también todos sus componentes gráficos y visuales externos (la carpeta oculta *frontend/dist*), solucionando definitivamente el error de dependencias.

Por otro lado, aunque se meten los *.py* y el *.css* dentro del ejecutable, **la base de datos (.db) siempre debe quedarse fuera**.

Si se metiera la base de datos dentro del *.exe*, al abrir el programa se podrían crear técnicos y tickets perfectamente, pero al cerrar el programa, **Windows borraría la carpeta temporal donde vuelca los archivos desempaquetados y se perderían todos los datos**.

Por eso, en aplicaciones con bases de datos locales (*SQLite*), la estructura profesional más habitual es tener una carpeta final que contenga:

- El ejecutable *.exe* (con los *.py* y el **diseño** ocultos dentro).
- El archivo *.db* (fuera del *.exe*, para que los datos sean permanentes y no se borren).

Hito 7 – Compilación definitiva:

Tras aplicar la doble solución del Hito 6, se ejecutó por última vez el script “*compilar.py*”. Esta vez, al abrir el ejecutable resultante, el servidor interno de *Streamlit* arrancó sin errores, encontró todas sus dependencias, cargó la base de datos local y mostró la interfaz gráfica en el navegador a la perfección.

3.2 CÓDIGO DE LOS NUEVOS ARCHIVOS PARA AUTOMATIZAR LA COMPILACIÓN

> Recordatorio de por qué compilar a través de dos archivos “*launch_gestion_incidencias.py*” y “*compilar.py*”:

La necesidad de separar el proceso en estos dos archivos responde a cómo funciona la arquitectura de Streamlit combinada con PyInstaller:

¿Por qué “*launch_gestion_incidencias.py*”? **Streamlit** está diseñado para ejecutarse nativamente desde una terminal leyendo un archivo de script. Cuando se empaqueta una app para que sea un **.exe** de un solo archivo (**--onefile**), Windows descomprime todo su contenido en una carpeta temporal oculta (**_MEIPASS**). “*launch_gestion_incidencias.py*” actúa como el punto de entrada oficial, localizando dinámicamente dónde ha descomprimido Windows los archivos y le ordena a **Streamlit** que ejecute la aplicación desde esa ruta secreta. Además, al **importar los módulos del backend** al principio obliga a **PyInstaller** a rastrear y empaquetar todas las librerías necesarias.

¿Por qué “*compilar.py*”? escribir comandos kilométricos en la **terminal** de **comandos** expone el proceso a **errores de sintaxis**, conflictos con los **puntos y comas (;)** y problemas con las **rutas relativas**. “*compilar.py*” traslada esas instrucciones a un **script** de Python puro, permitiéndonos calcular con precisión matemática las rutas absolutas del disco duro (**os.path.abspath**) para asegurar que **PyInstaller** coja “*gestion_incidencias.py*” y “*style.css*” y los incruste dentro del ejecutable sin margen de error.

> **compilar.py**

```
#####
# IMPORTACIONES
#####
# Módulo principal de ejecución interno de la librería PyInstaller
import PyInstaller.__main__
# Módulo Operating System para comunicación con el SO en que se ejecutan la app
# realizar tareas de bajo nivel, especialmente la gestión de archivos, carpetas y rutas
import os

#####
# RUTAS
#####
# Calcula la ruta absoluta exacta de la carpeta donde está tu proyecto en el disco duro
directorio_actual = os.path.abspath(os.path.dirname(__file__))

# Construye las rutas absolutas completas para asegurar que PyInstaller las encuentre sí o sí
ruta_app = os.path.join(directorio_actual, 'gestion_incidencias.py')
ruta_css = os.path.join(directorio_actual, 'style.css')

#####
# LLAMADA A PYINSTALLER
#####
# Llama al motor de PyInstaller pasándole una lista con todas las instrucciones de empaquetado
PyInstaller.__main__.run([
    'launch_gestion_incidencias.py', # Archivo principal que arrancará el .exe
    '--onefile', # Agrupa todo el proyecto en un único archivo ejecutable
    '--windowed', # Oculta la consola negra de Windows al hacer doble clic
    '--collect-all', 'streamlit', # Empaqueta el motor, dependencias y componentes visuales de Streamlit
    '--collect-all', 'streamlit_option_menu', # Empaqueta el código y los recursos web (frontend) del menú
    '--add-data', f'{ruta_app};.', # Incrusta el archivo principal de la interfaz dentro del .exe (en la raíz '.')
    '--add-data', f'{ruta_css};.' # Incrusta la hoja de estilos CSS dentro del .exe
])
```

> **import PyInstaller.__main__**:

Importa el módulo principal de ejecución interno de la librería **PyInstaller**. Básicamente, sirve para **ejecutar PyInstaller de forma programada** desde dentro de un **script** de Python, en lugar de tener que teclear los comandos a mano en la terminal.

Al hacer esa importación se obtiene acceso a la función **.run()**; ésta acepta una lista de cadenas de texto y las procesa exactamente igual que si se hubieran escrito en la consola de comandos. Esta es la pieza clave del Hito 5 que nos permitió sortear el problema de las rutas: al usar la terminal para compilar, ésta intercepta lo que se escribe y los caracteres especiales (como el punto y coma ; de `--add-data "gestion_incidentes.py;"`) rompen el comando.

Al usar **import PyInstaller.__main__** y llamar a **PyInstaller.__main__.run([...])**, se le entrega la lista de instrucciones directamente al "cerebro" del compilador y se garantiza que todos los parámetros y rutas se lean de forma literal y segura.

> **import os**:

Importa el módulo **"Operating System"** (Sistema Operativo), que es una de las librerías estándar más importantes de Python. Su función principal es permitir que el código de Python se comunique directamente con el sistema operativo (Windows, Mac o Linux) en el que se está ejecutando para realizar tareas de bajo nivel, especialmente la **gestión de archivos, carpetas y rutas**.

En el proyecto ha permitido resolver el problema de encontrar los archivos ocultos y empaquetar la aplicación correctamente, habiéndose empleado en tres archivos clave (**compilar.py**, **launch_gestion_incidentes.py** y **gestion_incidentes.py**) principalmente para usar las herramientas de **os.path**:

- **Calcular rutas absolutas automáticamente**

PyInstaller daba errores porque no sabía dónde estaban los archivos. Con **os** se pudo solucionar esto matemáticamente:

- **os.path.dirname(__file__)**: pide a Python que recorte el nombre del archivo y se quede solo con la carpeta donde está guardado.
 - **__file__**: variable interna y especial oculta de Python. Contiene automáticamente el nombre o la ruta del propio archivo que se está ejecutando en ese preciso momento.
 - **os.path.dirname()**: toma una ruta y recorta el nombre del archivo final, devolviendo únicamente la ruta de la carpeta que lo contiene.
- **os.path.abspath(...)**: le pide a Python la **ruta completa y exacta desde el disco duro (C:)** hasta el archivo/directorio que está leyendo; transforma cualquier ruta (relativa, incompleta o desordenada) en una ruta absoluta completa y normalizada. El término **"abspath"** es simplemente la abreviatura de **Absolute Path** (Ruta Absoluta).

Esto permitió que el script **"compilar.py"** supiera exactamente dónde estaba el proyecto sin que tener que escribir la ruta a mano (lo cual habría dado error si se movía la carpeta a otro sitio).

- **Unir rutas de forma segura (os.path.join)**

Esta herramienta permite que la aplicación sea universal; encargada de unir o empalmar diferentes fragmentos de texto para construir una ruta de archivo completa y válida.

En el código se usan funciones como **os.path.join(directorio_actual, 'gestion_incidentes.py')** u **os.path.join(sys._MEIPASS, archivo)**.

¿Por qué usar esto en lugar de escribir simplemente la ruta con texto (directorio + "\\gestion_incidentes.py")?

- Windows utiliza barras invertidas para las carpetas (C:\Usuarios\Carpeta).
- Mac y Linux utilizan barras normales (/Users/Carpeta).

Al usar **os.path.join**, el módulo **os** detecta automáticamente en qué sistema está funcionando el código y **pone la barra correcta**, logrando que el código esté preparado para ser compilado en un Mac sin tener que cambiar ni una sola línea.

> `PyInstaller.__main__.run`

La instrucción `PyInstaller.__main__.run([...])` es la función que realmente activa la compilación.

Normalmente, a un programa de terminal se le dan las órdenes escribiendo una frase larga separada por espacios, como: `pyinstaller --onefile --windowed mi_archivo.py`.

`.run()` acepta esa misma frase, pero **troceada en una lista de textos** (cadenas o *strings*). Toma cada parámetro como un elemento independiente dentro de unos corchetes [...].

Cuando Python lee esta línea, le entrega esa lista directamente al **núcleo** de `PyInstaller`, el cual procesa la orden exactamente igual que si se hubiera tecleado a mano y presionado *Enter*.

Esta función fue la pieza clave que resolvió definitivamente el problema de las rutas y los archivos perdidos, por dos grandes motivos:

- **Evitando la terminal:** al intentar usar `--add-data "gestion_incidentes.py;."` en la consola, Windows veía el punto y coma (;), creía que era una orden del sistema y rompía el comando. Al usar `.run()`, se entregan las instrucciones de forma interna. Para Python, ese punto y coma dentro de las comillas simples (';.') es un simple trozo de texto inofensivo. El motor de `PyInstaller` lo recibe intacto y entiende a la perfección que debe incrustar el archivo.
- **Inyección de variables:** al usar esta función, se pudieron introducir las **variables dinámicas** calculadas con `os` (las rutas absolutas). En lugar de escribir texto fijo, se pudo usar `f'{ruta_app};.'`, lo que permitió que Python calculara la ruta exacta del disco duro, le pegara el `;` al final, y se la inyectara directamente a `.run()`, logrando un empaquetado matemático y sin margen de error.

> **launch_gestion_incidencias.py**

```
#####
# IMPORTACIONES
#####

# Librería de Python que permite acceder a variables, parámetros y funciones específicos
# que interactúan de forma directa con el intérprete
import sys

# Módulo Operating System para comunicación con el SO en que se ejecutan la app
# realizar tareas de bajo nivel, especialmente la gestión de archivos, carpetas y rutas
import os

# cli: módulo de Streamlit que contiene la interfaz de línea de comandos (CLI)
# Se usa en scripts de Python para ejecutar aplicaciones de forma programada
from streamlit.web import cli as stcli

# Importación de los archivos de backend para obligar a PyInstaller a incorporarlos en el ejecutable
# Entrará a leerlos y empaquetará sqlite3 y cualquier otra librería que se use
import db
import models
import repository
import service

#####
# PUNTO DE ENTRADA
#####
# Punto de entrada > solo se ejecuta si se viene de doble clic o ejecución directa del archivo por el usuario
if __name__ == '__main__':
    # Comprobación de si se está dentro del .exe (la carpeta secreta _MEIPASS)
    # Herramienta nativa de Python "get attribute"
    if getattr(sys, 'frozen', False):
        # Busca dentro del módulo sys,
        # intenta encontrar una variable llamada 'frozen'
        # y, si por algún motivo no existe, no da error y devuelve False

        # Si es un .exe, se coge la ruta secreta absoluta
        carpeta_base = sys._MEIPASS
    else:
        # Si se está en un script normal, se coge la ruta de esta carpeta
        carpeta_base = os.path.dirname(os.path.abspath(__file__))

    # Ruta donde se encuentra el script de la interfaz de la app
    ruta_script = os.path.join(carpeta_base, "gestion_incidencias.py")

#####
# SIMULACIÓN DE EJECUCIÓN DE STREAMLIT DESDE TERMINAL
#####
# Se pasa la ruta absoluta al motor de Streamlit
# Simula la lectura de comando desde la terminal "streamlit run [ruta_absoluta]"
sys.argv = ["streamlit", "run", ruta_script, "--global.developmentMode=false"]
# stcli.main(): coge esas instrucciones y arranca el servidor de Streamlit
# sys.exit: asegura liberación de memoria y cerrado del .exe al cerrar la ventana de la app
sys.exit(stcli.main())
```

> import sys:

Importa el módulo **"System"** (Sistema), que es otra de las librerías estándar fundamentales de Python.

Mientras que el módulo `os` (que vimos antes) se encarga de hablar con el sistema operativo (Windows o Mac), el módulo `sys` se encarga de hablar directamente con el **intérprete de Python** (el "cerebro" interno que lee y ejecuta el código); es una librería del sistema interno de Python.

Le permite al programa acceder a **variables de configuración profunda** y manipular **cómo interactúa con el entorno** que lo está ejecutando (Windows en este caso).

Este módulo es el **protagonista principal del archivo "launch_gestion_incidencias.py"**. Sin él no sería posible el "engaño" para que **Streamlit** funcionara dentro de un `.exe`. Se ha empleado en 4 puntos fundamentales:

- **El detector de ejecutables (`sys.frozen`) > `if getattr(sys, 'frozen', False):`:**

Cuando **PyInstaller** compila la app y la convierte en un `.exe`, le **inyecta en secreto una variable llamada `frozen`** (congelado) al módulo `sys` (inyecta `sys.frozen = True`).

En el mundo de Python:

- En el caso de programas con archivos `.py` se habla de código "**líquido**", puesto que puede modificarse, añadirle librerías y cambiar cosas sobre la marcha.
- Mientras que al proceso de empaquetar un programa con **PyInstaller** se le conoce técnicamente como **freezing** (congelar); al pasar **PyInstaller**, se toma todo el código, el motor de Python y las librerías y se "congela" todo en un bloque: un único archivo `.exe`. Ese archivo ya no necesita que Python esté instalado en el ordenador y nadie puede modificar su código interno.

Con esta línea, el script le está preguntando al cerebro de Python si se está ejecutando como un archivo `.exe` normal o si es un ejecutable congelado:

- Escenario A (el **código líquido**): cuando se ejecuta el código dándole al botón de ejecutar el código normal (`.py`), la variable secreta `frozen` no existe. El código entiende que NO está congelado y sabe que se está en fase de desarrollo.
- Escenario B (El **bloque de hielo**): el cliente final hace **doble clic** en el archivo `.exe`. Al arrancar, **PyInstaller** (que está oculto dentro del archivo) **inyecta** la variable `frozen`. El código la detecta, entiende que SÍ está congelado, y cambia su comportamiento para buscar los archivos en la carpeta oculta temporal de Windows (`_MEIPASS`).

Esto es lo que permite que el código tome decisiones distintas dependiendo de si se está probando la app o si el cliente final la está usando.

¿Qué hace `getattr(sys, 'frozen', False)`?:

- La función `getattr()` es una herramienta nativa de Python que significa "**get attribute**" (obtener atributo), y siempre recibe tres instrucciones: `getattr(objeto, 'nombre_del_atributo', valor_por_defecto)`.
- En el código se le está pidiendo a Python que busque dentro del módulo `sys` la variable '`frozen`' y, si por algún motivo no existe, no dé un error y devuelva simplemente `False`.

¿Por qué no se escribe directamente un `if sys.frozen`?:

El módulo `sys` es el estándar del ordenador y **no tiene ni idea de lo que es la variable `frozen`**.

- Si se hubiera escrito el código de forma directa (`if sys.frozen:`), el programa probaría a leerlo, vería que no existe y **se estrellaría instantáneamente** devolviendo un error del tipo `AttributeError`. No se podría volver a probar el código nunca más sin compilarlo.
- `getattr(sys, 'frozen', False)` actúa como un escudo: Python busca la variable; al no encontrarla durante las pruebas, simplemente toma el valor `False` y sigue funcionando tranquilamente.

- **La ruta a la carpeta secreta (`sys._MEIPASS`) > `carpeta_base = sys._MEIPASS`:**

Si el paso anterior detecta que sí es un ejecutable congelado, **PyInstaller** crea automáticamente esta variable especial dentro de `sys`. Contiene la ruta exacta a esa **carpeta oculta y temporal de Windows** donde se acaba de descomprimir la aplicación (el `_MEIPASS` referido en el Hito 4). Gracias a esto, sabe dónde buscar los archivos.

- El simulador de la terminal (`sys.argv`) > `sys.argv = ["streamlit", "run", ruta_script, "--global.developmentMode=false"]`: `sys.argv` (**Argument Vector**) es **la lista interna donde Python guarda las palabras que el usuario escribe en la consola** negra. Como al hacer doble clic en el `.exe` el usuario no escribe nada, se usa esta línea para **reescribir la realidad**. Se mete a la fuerza esa lista de palabras al cerebro de Python. Así, cuando **Streamlit** arranca un milisegundo después, mira el `sys.argv`, cree que alguien ha escrito el comando en una consola y enciende el servidor web.

>> Se le está inyectando a la fuerza a Python la instrucción para que se comporte como si alguien hubiera abierto una consola de comandos y hubiera escrito **`streamlit run gestion_incidentes.py`**.

> > `--global.developmentMode=false` es opcional pero muy recomendable, sirve para ocultar el botón de "Deploy" y las herramientas de desarrollador en la esquina superior derecha de la app, dándole un aspecto más finalizado.

- El cierre limpio (`sys.exit()`) > `sys.exit(stcli.main())`: Sirve para obligar al programa a cerrar todo y apagarse de forma segura. Se usa envolviendo el **comando de arranque** de **Streamlit** (`stcli.main()`) para garantizar que, cuando el usuario cierre la pestaña del navegador o la ventana de la app, el servidor interno de Streamlit se apague correctamente, liberando la memoria y no deje procesos fantasma colgados en Windows.

> `carpeta_base = os.path.dirname(os.path.abspath(__file__))`

De nuevo aquí se combinan las dos herramientas previamente analizadas (`os.path.abspath` y `os.path.dirname`).

Esta línea averigua **la ruta absoluta, completa y exacta de la carpeta donde está guardado este archivo que está leyendo en ese instante y la guarda en la variable `carpeta_base`**:

- `__file__` (**El núcleo**): Python identifica el archivo actual (por ejemplo: `launch_gestion_incidentes.py`).
- `os.path.abspath(...)` (**La capa intermedia**): coge ese nombre y lo transforma en una dirección desde la raíz del disco duro (ejemplo: `C:\Usuarios\TuNombre\Proyectos\Incidentes\launch_gestion_incidentes.py`).
- `os.path.dirname(...)` (**La capa exterior**): coge esa ruta larga y recorta el nombre del archivo final, dejando únicamente la ruta limpia de la carpeta: `C:\Usuarios\TuNombre\Proyectos\Incidentes`.
- `carpeta_base = ...`: guarda esa ruta final en una variable para poder usarla después.

Esta línea se ejecuta en la opción B antes indicada (cuando el bloque else entra en acción), al no tratarse del caso de un ejecutable, sino de estar corriendo código `.py` en PyCharm. Es imprescindible porque cuando se está programando, la carpeta temporal secreta de Windows no existe. Con esta línea, el script recalcula matemáticamente dónde está la carpeta real del proyecto en el ordenador.

> `from streamlit.web import cli as stcli`:

Normalmente, cuando se usa **Streamlit**, se abre una terminal y se escribe **`streamlit run`** Al hacer eso, por debajo se está llamando a la **Interfaz de Línea de Comandos (CLI)** de **Streamlit**. Con esta línea, se importa el motor interno de arranque de **Streamlit** directamente al código de Python para poder encenderlo desde éste, sin necesidad de usar una terminal.

Es por tanto, un módulo interno de Streamlit que contiene la interfaz de línea de comandos (CLI). Se usa en scripts de Python para ejecutar aplicaciones de forma programada.

3.3 RESULTADO PARA DISTRIBUCIÓN

Tras ejecutar

```
python compilar.py
```

se genera el ejecutable para Windows: contiene todo el código, incluyendo la llamada a `init_db()` para que el motor de **SQLite** fabrique la base de datos "`gestionIncidencias.db`" desde cero.

- Por tanto, **para la distribución no es necesario mandar ningún archivo de base de datos**. Simplemente hay que tomar el archivo "`launch_gestion_incidencias.exe`", que se genera dentro de una carpeta "`dist`" que el IDE crea automáticamente, meterlo en un archivo comprimido (por ejemplo, un .zip) y enviarlo. Cuando el usuario lo descomprima en una carpeta de su ordenador y haga doble clic, la aplicación arrancará y creará una base de datos virgen a su lado, en la misma carpeta donde se haya ubicado el `.exe`.
- **La advertencia de Windows Defender:** al compartir el `.exe` es seguro que **Windows Defender** (o el antivirus del usuario) haga saltar una pantalla azul de advertencia diciendo "Windows protegió su PC". Esto ocurre porque el programa no tiene una "**Firma Digital**" oficial de Microsoft. El usuario final tendrá que hacer clic en "**Más información**" y luego en "**Ejecutar de todas formas**".
- **Actualizaciones futuras:** si el día de mañana se mejora el código y se quiere distribuir una nueva versión, solo hay que enviar el nuevo `.exe`. Los usuarios simplemente reemplazarán el ejecutable antiguo por el nuevo en su carpeta, y como la base de datos `.db` está fuera, conservarán todos sus tickets y técnicos intactos.

3.4 EJECUTABLES EN OTROS SISTEMAS OPERATIVOS

> **Mac**

REGLA DE ORO DE PyInstaller: no existe la compilación cruzada.

PyInstaller inspecciona el **sistema operativo** en el que está funcionando y empaqueta las piezas específicas de ese sistema. Por tanto, es matemáticamente imposible fabricar un ejecutable de Mac desde un ordenador con Windows.

Para lograrlo, el proceso es el siguiente:

- **Emplear un Mac físico (o una máquina virtual):** llevar todos los archivos de código fuente (los `.py`, el `.css`) a un sistema macOS.
- **Preparar el entorno:** en ese Mac, instalar Python y todas las librerías que usa el proyecto mediante la terminal (`pip install streamlit streamlit_option_menu pyinstaller`).
- **Ejecutar el compilador:**
 - **El código fuente es 100% compatible:** al emplear la librería `os` para gestionar las **rutas** (por ejemplo, con `os.path.join` en el archivo "`compilar.py`" y en "`launch_gestion_incidencias.py`" el código ya está preparado para Mac; si bien **Windows** usa barras invertidas (`\`) para las carpetas y **Mac** usa barras normales (`/`), Python detecta automáticamente en qué sistema está y pone la barra correcta, sin tener que modificar una sola línea de código.
 - Abrir la terminal del Mac, navegar hasta la carpeta del proyecto y ejecutar el script "`python compilar.py`".
- **Resultado:** **PyInstaller** generará en la carpeta `dist` un **archivo ejecutable de Unix (sin extensión)**. El usuario de Mac solo tendrá que hacer doble clic sobre él para que se abra la aplicación en su navegador, exactamente igual que en Windows.

Nota técnica para Mac: Apple ha cambiado recientemente de procesadores Intel a los suyos propios (**Apple Silicon**: M1, M2, M3). Si se compila la app en un Mac con procesador M1, funcionará perfectamente en otros Mac modernos, pero podría no abrirse en un Mac antiguo de hace 6 años con procesador Intel, y viceversa. Para asegurar la máxima compatibilidad en un entorno mixto, los desarrolladores suelen necesitar compilar la aplicación dos veces (una en un Mac con **Intel** y otra en un Mac con procesador M).

> Archivo requirements.txt

Se puede generar un archivo "**requirements.txt**" en Windows para que, al llevar el proyecto a otro ordenador, sin importar el sistema operativo, se **instalen todas las librerías automáticamente** sin tener que escribirlas a mano.

Pasos para crearlo para crearlo en Windows y usarlo en un Mac:

- **Generar el archivo en Windows**
- Abrir el proyecto en **PyCharm**.
- Abrir la **Terminal** integrada.
- Ejecutar el comando:

```
pip freeze > requirements.txt
```

- El comando **pip freeze** escanea el entorno virtual actual y hace un listado de todas las librerías que instaladas junto con sus versiones exactas (por ejemplo, streamlit==1.32.0).
- El símbolo **>** vuelca todo ese texto en un archivo nuevo llamado "**requirements.txt**" en la misma carpeta que el código.

Si se abre el archivo recién creado, se pueden observar un montón de librerías, muchas de ellas dependencias secundarias que **Streamlit** instala por defecto. Para que la lista sea lo más limpia posible de cara al Mac, se puede borrar todo el contenido y dejar únicamente las tres herramientas principales que realmente se necesitan en este caso (con o sin el número de versión). Así, el archivo puede dejarse tal y como:

```
streamlit  
streamlit-option-menu  
pyinstaller
```

Nota: No hace falta incluir **sqlite3** ni os ni sys porque ya vienen preinstaladas dentro del propio lenguaje Python.

Cuando se lleve la carpeta del proyecto al Mac el archivo debe ir dentro.

Una vez en el Mac:

- Abrir la carpeta del proyecto con el **PyCharm** del Mac.
- Abrir la **Terminal** integrada de PyCharm.
- Ejecutar este único comando:

```
pip install -r requirements.txt
```

Al usar el parámetro **-r** (de **read** o leer) Python lee el archivo y **PyCharm** descarga e instala automáticamente **Streamlit**, el **menú de opciones** y **PyInstaller**. Cuando termine, solo habrá que ejecutar **python compilar.py** y se generará la aplicación nativa para Mac lista para funcionar.

> Distribución del ejecutable generado en Mac

El ejecutable que genere en Mac es distribuible, pero hay un matiz de seguridad mucho más estricto que en Windows. Al enviarlo e intentar instalarlo en otros Mac pueden presentarse problemas con el guardián del sistema operativo de Apple: **Gatekeeper**:

- Apple es extremadamente restrictiva con los programas que se descargan de internet y no pasan por su tienda oficial (App Store).
 - Si se envía la app a otro usuario de Mac (por ejemplo, en un archivo .zip) y este intenta hacer doble clic para abrirla, Gatekeeper la bloqueará y mostrará un mensaje parecido a este: "No se puede abrir la aplicación porque proviene de un desarrollador no identificado" (o incluso, en versiones recientes, puede asustar al usuario diciendo que el archivo está dañado y sugiere moverlo a la papelera).
- Esto ocurre simplemente porque el programa es "anónimo" a los ojos de Apple.

Para distribuirlo hay 2 caminos:

> Distribución Interna o Privada (El método gratuito)

Si la app es para uso interno de la empresa, para su equipo de técnicos o para conocidos, se puede distribuir sin pagar nada. Solo hay que enseñarles a los destinatarios cómo darle permiso al Mac **la primera vez que la abran**:

- **El atajo rápido:** en lugar de hacer doble clic, el usuario debe hacer **clic derecho** (o mantener pulsada la tecla Control + clic izquierdo) sobre el ejecutable y seleccionar **Abrir**. Aparecerá una advertencia, pero esta vez incluirá un botón que permite **Abrir de todas formas**.
- **Desde los ajustes:** si el paso anterior falla, el usuario debe intentar abrir la app normalmente, y luego ir a **Ajustes del Sistema > Privacidad y seguridad**. Al bajar un poco, verá un mensaje indicando que se bloqueó la app y un botón para **"Abrir de todos modos"**.

Una vez que el usuario hace esto la primera vez, el Mac registra el archivo como "seguro" y, de ahí en adelante, se abrirá con un doble clic normal y corriente.

> Distribución Comercial (El método profesional y de pago)

Si el objetivo fuera vender el software o distribuirlo masivamente sin que los usuarios vean pantallas de advertencia, es obligado entrar en el ecosistema oficial de Apple. Esto implica:

- Pagar la inscripción al **Apple Developer Program** (unos 99€ anuales).
- **Firmar el código (Code Signing):** añadirle un certificado digital al ejecutable que dice "Yo soy el creador y Apple me conoce".
- **Notarización:** enviar la aplicación a los servidores de Apple para que un sistema automatizado la escanee en busca de virus y le dé un sello de aprobación para Gatekeeper.

> Linux

REGLA DE ORO DE PyInstaller: no existe la compilación cruzada.

Al igual que ocurre con **macOS**, no se puede generar un ejecutable nativo de Linux desde Windows. **PyInstaller necesita estar ejecutándose físicamente dentro de un entorno Linux** para poder empaquetar las librerías correctas de ese sistema.

Para lograrlo, el proceso es el siguiente:

- Emplear un **PC con Linux** (o una **máquina virtual/WSL**): llevar la carpeta con todos los archivos de código fuente (los **.py**, el **.css** y el archivo **"requirements.txt"**) a un sistema **Linux** (por ejemplo, Ubuntu). También sirve usar el **Subsistema de Windows para Linux (WSL)** integrado en el propio Windows 10/11.
- **Preparar el entorno:** a diferencia de Windows, Linux suele traer **Python preinstalado**. Solo hay que abrir la terminal de Linux, navegar a la carpeta del proyecto y ejecutar el comando para instalar las dependencias creadas previamente: **pip install -r requirements.txt**.
- **Ejecutar el compilador:**
 - El código fuente es 100% compatible: de nuevo, gracias al uso de la librería **os** y **os.path.join**, el código está preparado para las rutas de Linux (que al igual que Mac, utilizan la barra normal **/**). No hay que modificar absolutamente nada en el código.
 - Ejecutar el script desde la terminal de Linux con: **python compilar.py**.
- **Resultado:** PyInstaller generará en la **carpeta dist** un archivo ejecutable binario (formato **ELF**, también sin extensión, al igual que en Mac).

Nota técnica para Linux (La regla de la **compatibilidad hacia adelante**):

El mayor problema al compilar en Linux no es el tipo de procesador, sino una **librería** interna del sistema llamada **glibc**. Si se compila la app en una versión muy moderna de Linux (ej. Ubuntu 24.04), el ejecutable no funcionará en versiones más antiguas (ej. Ubuntu 20.04) porque le faltarán librerías actualizadas.

Por lo tanto, la **regla de oro** de los desarrolladores en Linux es: **"Compilar siempre en la versión más antigua** del sistema operativo al que se quiera dar soporte". Un ejecutable creado en un Linux antiguo funcionará perfectamente en uno moderno, pero nunca al revés.

> Distribución del ejecutable generado en Linux:

El ecosistema de Linux es de código abierto, por lo que no existen programas de pago para desarrolladores ni un "Gatekeeper" corporativo como en Apple. Sin embargo, Linux tiene un sistema de permisos de seguridad extremadamente estricto para evitar virus.

Cuando se envíe el ejecutable a otro usuario de Linux y este lo descargue, el sistema operativo lo tratará como un simple archivo de datos inofensivo. Si el usuario hace doble clic, no ocurrirá nada o se abrirá como un archivo de texto.

Para que la aplicación funcione, el destinatario debe **otorgarle permisos de ejecución la primera vez**. Hay dos formas de hacerlo:

- El **método gráfico** (Para usuarios normales):
 - Hacer **clic derecho** sobre el archivo descargado.
 - Seleccionar **Propiedades**.
 - Ir a la pestaña de **Permisos**.
 - Marcar la casilla que dice: **"Permitir ejecutar** el archivo como un programa".
 - Aceptar y hacer doble clic. La app abrirá el navegador mostrando tu interfaz de Streamlit.
- El **método por terminal** (Para usuarios **avanzados**):
 - Abrir la **terminal** en la carpeta donde está el archivo.
 - Escribir el comando: **chmod +x nombre_del_archivo**
 - Ejecutarlo escribiendo: **./nombre_del_archivo**

Una vez que se le ha dado el permiso de ejecución, el archivo se comportará como un programa normal y corriente para siempre en ese ordenador.

ANEXO

**Análisis en detalle
de los archivos de código
de la interfaz gráfica
de la Versión 2**

A-1. `gestion_incidencias.py`

```
pip install streamlit-option-menu
from streamlit_option_menu import option_menu
```

> **Streamlit-option-menu** es un componente **Streamlit** sencillo que permite a los usuarios seleccionar un único elemento de una lista de opciones en un menú.

Streamlit es fantástico para prototipar rápido, pero es muy rígido a la hora de cambiar el diseño visual. Por esto, se importa esta dependencia para el **diseño de las pestañas** de la app, en lugar de emplear las opciones nativas de **Streamlit** como **st.tabs** o **st.sidebar.radio**, permitiendo mayor control de este:

> Control de estilos:

Con el uso de **st.tabs** el diseño era básico y gris.

El componente **option_menu** permite inyectar un **diccionario** de configuración donde se definen exactamente los colores, bordes y comportamientos del menú.

Gracias a esto se fuerzan los colores de las pestañas y se permite diferenciar el aspecto de la pestaña seleccionada frente a las demás.

> Integración directa de iconos:

Permite, además, pasar una lista directa de iconos (**icons=["card-list", "person-badge", "gear"]**) que se renderizan automáticamente y con un aspecto muy profesional al lado del texto de cada sección. Con **pestañas nativas**, sin embargo, se requiere inyectar **emojis de texto** que no siempre se ven iguales en todos los navegadores o sistemas operativos.

> Integración / orientación:

Al configurarlo con **orientation="horizontal"** pasa a funcionar como una barra de navegación superior clásica de cualquier página web moderna.

> Sinergia con el style.css:

Por otro lado, para lograr el **efecto visual** de "carpeta" en el que la pestaña se fusiona visualmente con el recuadro gigante de abajo se necesitaba manipular los márgenes inferiores (**margin-bottom: -2px**) y las posiciones Z (**z-index: 10**) de la barra de navegación.

Las herramientas nativas de **Streamlit** cambian los nombres de clase internos en cada actualización, lo que puede romper el archivo **CSS** constantemente. **option_menu** aísla ese comportamiento y garantiza que el diseño no se rompa en el futuro.

Si el **diseño estético** no fuera una prioridad y solo importara la funcionalidad bruta se usaría **st.tabs** sin más. Pero para el acabado visual estricto y profesional esta librería era la herramienta adecuada para el trabajo.

```
@st.dialog("⚠️ Confirmar Baja de Técnico")
```

> Decorador **@st.dialog** de **Streamlit** introducido recientemente, que **transforma una función normal de Python en una ventana modal** o ventana emergente (**pop-up**).

Al colocar **@st.dialog("Título de la ventana")** justo **encima de la definición de una función** se cambia por completo el comportamiento de esa función. En lugar de imprimir su contenido en el flujo normal de la página, **Streamlit** hace lo siguiente:

- > **Bloquea el fondo:** oscurece visualmente el resto de la aplicación y la desactiva temporalmente, impidiendo que el usuario haga clic en otras pestañas o botones.
- > **Crea una capa superpuesta:** abre un recuadro flotante en el centro de la pantalla con el título que se ha puesto al decorador.
- > **Aísla la interacción:** todo lo programado dentro de esa función (textos, botones, selectores) se dibujará exclusivamente dentro de esa ventana.

En este caso se usa para **prevenir errores accidentales**; se evita la baja accidental de usuarios ante clics por error en el botón "**Dar de Baja**". Con esta lógica dentro de una función decorada con **@st.dialog** se logra que el botón de la interfaz no borre nada, sino que simplemente invoque o llame a la ventana modal y se obliga al usuario a detenerse, leer la advertencia roja y **confirmar** su decisión haciendo clic en un segundo botón ("**SI**") dentro del **pop-up** aislado antes de modificar la base de datos.

```
st.rerun()
```

> Es una instrucción fundamental para entender cómo funciona la arquitectura de **Streamlit**; interrumpe inmediatamente la ejecución actual y **volver a ejecutar todo el script de Python** desde la línea 1 hasta el final.

> ¿Por qué es **imprescindible** en el gestor de incidencias?:

Streamlit no funciona como otras librerías tradicionales; **no actualiza partes individuales de la pantalla**. Dibuja la interfaz leyendo el código de arriba a abajo en cada interacción.

> Sin **st.rerun()** en los formularios, ocurriría lo siguiente:

- Un usuario rellena el formulario de "Nuevo Técnico" y pulsa el botón.
- La capa de servicios guarda correctamente ese nuevo empleado en la **base de datos SQLite**.
- El botón termina su ejecución y muestra el **mensaje de éxito**.
- El **problema**: la tabla que muestra la plantilla de técnicos seguiría mostrando los datos antiguos, porque se dibujó en la pantalla antes de que se pulsara el botón y se actualizara la base de datos. Para ver al nuevo técnico, el usuario tendría que **recargar la página web a mano** (F5).

> Con **st.rerun()** justo después de realizar cualquier operación válida (crear un ticket, asignar un técnico o darlo de baja) se seguiría el siguiente flujo:

- **Acción**: el usuario pulsa un botón (ej: "Finalizar Ticket").
- **Persistencia**: la aplicación actualiza la información en la **base de datos** mediante tus reglas de negocio.
- **Aviso**: se muestra el mensaje de éxito en pantalla (**st.success**).
- **Reinicio**: se ejecuta **st.rerun()**: el programa se detiene en seco en esa línea y vuelve a empezar desde el principio; al volver a empezar, vuelve a consultar la base de datos. Como la base de datos ya tiene la información nueva, **la pantalla se redibuja con las tablas 100% actualizadas**.

```
except ValueError as e:
    st.error(str(e))
```

> Captura del error (**except ValueError as e**):

En "**service.py**" se configuraron reglas estrictas. Por ejemplo, si alguien intenta registrar un técnico sin poner un email válido, el código hace un "**raise ValueError('ATENCIÓN: El email del técnico debe contener el carácter '@'.')**".

Ese **raise** (levantar) lanza una alarma de emergencia. Si nada apaga esa alarma, la aplicación entera se bloquea y se rompe.

Aquí es donde muestra su funcionalidad "**except ValueError as e**:" en el archivo "**gestion_incidencias.py**": captura el error y lo almacena en una variable llamada **e** para examinarlo antes de que rompa el programa.

> Muestra el mensaje (**st.error(str(e))**):

Una vez que atrapado el error de forma segura en la variable **e** se avisar al usuario de lo que ha hecho mal:

- **str(e)**: extrae el texto escrito dentro del error en "**service.py**" (en este caso, el texto sobre la falta del '@').
- **st.error(...)**: es el comando nativo de **Streamlit** que dibuja un recuadro rojo brillante en la interfaz con un icono de advertencia.

> Gracias a estas dos líneas de código, cuando un usuario introduce datos inválidos o intenta asignar un ticket a un técnico inactivo, el sistema no se cuelga ni muestra una pantalla de error indescifrable. Simplemente **detiene la acción y le muestra un recuadro rojo** educado con las instrucciones exactas de lo que debe corregir para poder continuar.

```
# Definición de una función para llevar a cabo la carga del archivo
def load_local_css(file_name):
    try:
        # Apertura y solo lectura del archivo; guardado en una variable del texto contenido
        # with open(obtener_ruta(file_name), "r", encoding="utf-8") as f:
        # markdown carga el texto en la página entre las etiquetas <style>
        st.markdown(f'<style>{f.read()}</style>', unsafe_allow_html=True)
    # Manejo de error en caso de no encontrar el archivo
    except FileNotFoundError:
        st.warning(f"No se encontró el archivo '{file_name}'.")

# Ejecución de la función para cargar el archivo (pasado como parámetro)
load_local_css("style.css")
```

> Este bloque conecta la **lógica de Python** con el **diseño visual** definido en el archivo **style.css**.

Por defecto, **Streamlit** no tiene una forma oficial y directa de cargar un archivo **.css** externo de manera limpia. Para saltar esa limitación se crea esta **función que lee el archivo de estilos y lo inyecta a la fuerza en el código fuente de la página web**.

> Definición de la función > **def load_local_css(file_name):**

Se crea una función reutilizable "**load_local_css**" que recibe un único parámetro: el nombre del archivo a cargar ("**style.css**").

> Manejo de excepciones/errores (try / except)

```
try:
    ...
except FileNotFoundError:
    st.warning(f"No se encontró el archivo '{file_name}'.")
```

Se pide a Python que intente abrir el archivo. Si por algún motivo se ha movido o borrado por accidente, Python lanzaría un error fatal (**FileNotFoundError**) y la aplicación entera dejaría de funcionar. Al atraparlo con el **except**, se evita que la app se rompa y, en su lugar, se muestra una **advertencia amarilla** en pantalla (**st.warning**) avisando de que faltan los estilos.

> Lectura segura del archivo > **with open(file_name, "r", encoding="utf-8") as f:**

El comando **with open(...)** es la forma más segura de trabajar con archivos en Python:

- **"r"**: significa **read** (modo lectura); solo leer el archivo, no modificarlo.
- **encoding="utf-8"**: sistema de caracteres.
- El uso de **with** garantiza que, una vez termine de leer el archivo, Python lo cerrará automáticamente liberando memoria, incluso si ocurre algún error en el proceso.
- La lectura del archivo se guarda en una variable **f**.

> La inyección (**unsafe_allow_html**) > **st.markdown(f'<style>{f.read()}</style>', unsafe_allow_html=True):**

st.markdown muestra un string formateado como Markdown.

Esta es la línea más importante de todo el bloque. Hace 3 cosas simultáneamente:

- **f.read()**: coge todo el texto del archivo **style.css** y lo convierte en una sola cadena de texto gigante.
- **f'<style>...</style>'**: envuelve todo el texto de CSS dentro de las etiquetas estándar de **HTML <style>**, que es como los navegadores web entienden que lo que hay dentro son reglas de diseño.
- **unsafe_allow_html=True**: por motivos de ciberseguridad, la función **st.markdown** de Streamlit **bloquea y neutraliza cualquier código HTML** que se intente imprimir (para evitar ataques de inyección de código). Con el parámetro en **True** se indica a **Streamlit** que levante la seguridad y renderice ese código HTML exactamente como se le pasa; así se logra que el CSS se aplique al navegador.

> Ejecución > **load_local_css("style.css")**

Esta línea ejecuta la función pasándole el nombre del archivo de diseño, desencadenando todo el proceso justo al inicio del script para que los estilos estén listos antes de dibujar los botones y las tablas.

```
# =====
# 4. CABECERA
```

```
# =====  
# Título de la APP, equivalente a un h1  
st.title(" 🚚 SISTEMA DE GESTIÓN DE INCIDENCIAS")  
# Línea divisoria horizontal  
st.markdown("<hr>", unsafe_allow_html=True)
```

> Bloque de código que **dibuja la cabecera superior** de la página web.

> El título principal (**st.title**) > **st.title(" 🚚 SISTEMA DE GESTIÓN DE INCIDENCIAS")**:

Función **nativa** de **Streamlit** para crear el **encabezado principal** de la página (lo que en código web **HTML** se conoce como la etiqueta **<h1>**). Imprime el texto y el emoji en la parte superior.

Sin embargo, lo verdaderamente interesante ocurre con la conexión con el archivo **"style.css"** donde se programa una **regla** específica **para todos los elementos h1** de la aplicación. Así, el navegador le aplica un fondo de color gris claro, un borde definido, bordes redondeados y una pequeña sombra. El resultado es que el título parece un recuadro o **"tarjeta"** independiente.

> La línea separadora (st.markdown) > **st.markdown("<hr>", unsafe_allow_html=True)**:

Esta línea actúa como frontera visual, separando claramente la zona del título de la zona interactiva (las pestañas de navegación) que se dibujan inmediatamente debajo.

En **HTML**, la etiqueta **<hr>** (Horizontal Rule) dibuja una línea divisoria horizontal de lado a lado de la pantalla.

Mediante **st.markdown** se inyecta el código (la etiqueta); con el parámetro **unsafe_allow_html=True** se evita que **Streamlit** bloquee la etiqueta HTML, lo cual hace por defecto.

A su vez, el **"style.css"** también tiene **instrucciones** estrictas para esta **línea**; en lugar de la línea fina y gris por defecto que traen los navegadores se le da un grosor de 3 píxeles, un color azul pizarra (#64748b) y unos márgenes de separación muy concretos.

```
# =====  
# 5. MENÚ DE NAVEGACIÓN ("PESTAÑAS")
```

```
# =====
selected = option_menu(
    # Devuelve el texto de la selección y lo guarda
    # en la variable selected

    # Oculta el título del Menú (ya tenemos el st.title)
    menu_title=None,
    # Textos de las pestañas
    options=["Ver Incidencias", "Técnicos", "Gestión y Asignación"],
    # Iconos de la librería Bootstrap Icons
    icons=["card-list", "person-badge", "gear"],
    # Pestaña por defecto al abrir app > 0 = la primera
    default_index=0,
    # Fuerza a mostrar el menú en horizontal (no vertical) = barra de pestañas
    orientation="horizontal",
    # Diccionario de estilos
    styles={
        # Contenedor principal de las pestañas
        "container": {
            "padding": "0",
            "background-color": "#c8d6e5",
            "border": "none",
            "box-shadow": "none",
            "max-width": "100%",
            "margin-bottom": "-2px",
            "position": "relative",
            "z-index": "10"
        },
        # Envoltura de todas las pestañas
        "nav": {
            "background-color": "#c8d6e5",
            "justify-content": "flex-start",
            "margin": "0",
            "padding": "0"
        },
        # Cada elemento / pestaña de navegación
        "nav-item": {
            "flex": "0 0 auto",
            "margin-right": "5px"
        },
        # Zona interactiva (de clic) real en la pestaña (todas)
        "nav-link": {
            "font-size": "1.05rem",
            "font-weight": "600",
            "color": "#475569",
            "background-color": "#e2e8f0",
            "border": "1px solid #64748b",
            "border-bottom": "1px solid #64748b",
            "border-radius": "8px 8px 0 0",
            "padding": "10px 20px"
        },
        # Zona interactiva (de clic) real en la pestaña (la seleccionada)
        "nav-link-selected": {
            "background-color": "ffffff",
            "color": "000000",
            "border-bottom": "2px solid #f1f5f9",
            "font-weight": "700"
        }
    }
)
```

> Este bloque crea la **barra de navegación superior** (las **pestañas**) y “recuerda” en qué pestaña ha hecho clic el usuario para que el resto del programa sepa qué contenido debe mostrar.

Se distinguen 2 partes: la **configuración básica** y el **diseño visual**.

>> La Lógica y Configuración Básica:

- > **`selected = option_menu(...)`**: cada vez que el usuario hace clic en una pestaña, esta función devuelve el texto exacto de esa pestaña y lo guarda en la variable **`selected`**. Más abajo en el código se usan condicionales (`if selected == ...:`) para mostrar un contenido u otro.
- > **`menu_title=None`**: oculta el título principal del menú para que no ocupe espacio innecesario, ya que ya existe el título gigante "SISTEMA DE GESTIÓN...".
- > **`options=[...]`**: lista con los textos literales que se muestran en cada pestaña.
- > **`icons=[...]`**: asigna un icono de la librería estándar ***Bootstrap Icons*** a cada pestaña de la lista anterior, en el mismo orden.
- > **`default_index=0`**: le indica al programa que, al abrir la aplicación por primera vez, la **pestaña activa por defecto** debe ser la primera de la lista (índice 0, es decir, "Ver Incidencias").
- > **`orientation="horizontal"`**: fuerza a que el menú se dibuje de izquierda a derecha en lugar de apilarse de arriba a abajo. Esto es lo que lo convierte en una **barra de pestañas** en lugar de un menú lateral.

>> El diseño visual > Diccionario de Estilos (`styles={...}`)

El componente **`streamlit_option_menu`** se encuentra dentro de un **`iframe`** (una caja aislada), por lo que necesita que se le inyecte su propio CSS directamente a través de este diccionario de Python.

Streamlit trae muchos botones y tablas "de fábrica" (nativos).

Sin embargo, **`streamlit_option_menu`** no fue creado por los desarrolladores de ***Streamlit***, sino por un miembro de la comunidad. Por motivos estrictos de seguridad, cuando se usa un **componente que no es oficial, *Streamlit* no lo mezcla con el resto del código web**. En su lugar, crea un **`iframe`**, que es literalmente, una página web incrustada dentro de otra página web.

El archivo **`style.css`** se carga en la página web principal (la aplicación "padre"). Sin embargo, las reglas de diseño web dictan que el CSS de una página web principal no puede atravesar la barrera de un **`iframe`** para modificar lo que hay dentro. Si se intenta cambiar el color del menú usando el **`style.css`**, el navegador chocaría contra el muro del **`iframe`** y el menú seguiría viéndose con sus colores por defecto, ignorando las órdenes.

Solución: el **diccionario de Python**

El creador de **`streamlit_option_menu`** inventó una "puerta trasera" para el diseño; añadió el parámetro **`styles={...}`** a su código Python.

Al ejecutar la **`app`**:

- Se escriben los colores y bordes en ese diccionario de Python.
- ***Streamlit*** lee ese diccionario antes de construir el **`iframe`**.
- El componente coge esos datos, los traduce a lenguaje ***CSS*** internamente y los inyecta por dentro del **`iframe`** en el mismo momento en que lo está construyendo.

Es la única forma técnicamente posible de darle diseño personalizado a esa barra de navegación y hacer que se fusione visualmente con el resto del gestor de incidencias.

Aquí es donde se logra el aspecto de "carpeta física":

> **`"container"`** (El contenedor principal):

`"margin-bottom": "-2px"` y `"z-index": "10"`: hacen que toda la barra de pestañas baje 2px y se superponga por encima del recuadro gigante de contenido que hay justo debajo. Esto borra la línea de separación y hace que parezca una única carpeta unida.

`"background-color": "#c8d6e5"`: se funde exactamente con el azul de fondo de la app para que el contenedor en sí sea invisible.

> **`"nav"`** y **`"nav-item"`**:

`justify-content: flex-start`: alinea las pestañas a la izquierda

`margin-right: 5px`: deja un pequeño margen de 5px a la derecha de cada una, para que no se peguen como un solo bloque.

`"nav-link"` (El aspecto de las **pestañas INACTIVAS**):

"border-radius": "8px 8px 0 0": redondea las esquinas superior izquierda y superior derecha (8px), pero deja las esquinas inferiores completamente cuadradas (0) para que se peguen bien al recuadro inferior. Tienen un color grisáceo (#e2e8f0) para indicar que están en segundo plano y se pueden clicar.

"nav-link-selected" (El aspecto de la **pestaña ACTIVA**):

Al hacer clic en una pestaña, cambia automáticamente a este estilo.

"background-color": "#ffffff": se vuelve blanca puro contrastando con las pestañas inactivas.

"border-bottom": "2px solid #f1f5f9": "borra" la línea inferior de la pestaña activa pintándola del mismo color que el fondo, logrando la ilusión óptica de que **la pestaña "fluye" hacia dentro de la página**, como las carpetas de archivos de un archivador real.

Para entender qué son exactamente **nav** y **nav-item**, hay que tener en consideración cómo se construyen las páginas web por debajo, ya que el componente **option_menu** utiliza la terminología estándar de HTML y un popular sistema de diseño llamado **Bootstrap**.

> **nav** (La Barra de **Navegación**)

La palabra nav es la abreviatura de Navigation (Navegación). En el mundo web, representa la caja principal agrupa todos los enlaces.

¿Qué es físicamente? Es el contenedor horizontal entero que envuelve a todas las pestañas.

En el código, escribir que tenga **justify-content: flex-start** se le indica que empuje hacia la izquierda de la pantalla todos los elementos que tenga dentro. También se le puso el fondo del mismo color azul (#c8d6e5) para que esta "caja" sea invisible y se funda con el fondo de la app.

> **nav-item** (El Contenedor Individual)

La palabra nav-item significa **Elemento de Navegación**. Es cada una de las ranuras o de los espacios invisibles reservado para cada pestaña específica ("Ver Incidencias", "Técnicos", etc.).

En el código se le aplica **margin-right: 5px**; sin él, los elementos individuales chocarían entre sí y las tres pestañas parecerían un solo bloque gigante de texto. Al dar el margen se obliga a que cada pestaña se separe 5px de la que tiene a su derecha, creando ese efecto de "carpetas independientes".

Diferencia clave con nav-link

Es importante no confundir **nav-item** con **nav-link**:

nav-item es la "caja invisible" o ranura que reserva el espacio.

nav-link es el botón real, el color de fondo gris o blanco, el texto, el icono y lo que el usuario realmente pulsa.

Por eso se separan los estilos: al **nav-item** se le indica cómo separarse de los demás, y al **nav-link** se le dan qué colores y bordes debía tener.


```
# Contenedor grande que contiene todas las operaciones/opciones
with st.container():
    # Inyección de un identificador del contenedor para darle estilo en CSS
    st.markdown("<div id='recuadro-principal'></div>", unsafe_allow_html=True)
```

Este bloque de código es el responsable de crear el recuadro blanco gigante (la "**carpeta**") donde se **ubica todo el contenido interactivo** y lograr que se fusione con las pestañas superiores.

> Elemento agrupador > **with st.container()**:

La función nativa **st.container()** crea un **bloque invisible que sirve para agrupar elementos**.

Al usar la palabra **with** se indica a Python que meta en esta "caja madre" todo lo escrito en adelante con sangría (los formularios, las tablas, los textos).

Problema de **st.container()**: **Streamlit no permite asignarle un nombre**, una **clase** o un **ID** directamente. Por defecto, es una caja fantasma que el CSS no puede ver ni tocar.

> Identificación del contenedor > **st.markdown(...)**: como no se puede nombrar al contenedor **se inyecta un trozo de HTML** puro: **<div id='recuadro-principal'></div>**, que es un elemento div completamente vacío.

El **unsafe_allow_html=True** hace que **Streamlit** lo procese como código web real.

Este **div**, al estar vacío, no ocupa espacio ni dibuja nada en la pantalla; funciona exclusivamente como un **localizador** o "baliza oculta" dentro de ese contenedor agrupador.

> Conexión con el **style.css**:

En el **style.css** se escribe una regla que indica al navegador web que busque por toda la página el contenedor gigante de **Streamlit** que responda a **#recuadro-principal**; esto se logra con el **selector CSS**: **has(...)**.

Una vez que el navegador localiza ese contenedor exacto el **CSS** le aplica todo el **diseño** de golpe:

- Fondo blanco (#f1f5f9), borde gris (#64748b), esquinas inferiores redondeadas (border-radius: 0 8px 8px 8px) y le añade una sombra suave. 0
- Finalmente, ejecuta la orden de **tirar de toda la caja hacia arriba** (**margin-top: -38px**); hace que la caja suba, tape el hueco vacío y se deslice justo por debajo de la barra de pestañas, creando el **efecto visual de que las pestañas y el recuadro son una sola pieza sólida**. Sin este bloque exacto, el contenido de las pestañas simplemente flotaría sobre el fondo azul general de la aplicación de manera desordenada.

```
# -----
# PESTAÑA 1: VER INCIDENCIAS
```

```
# -----
if selected == "Ver Incidencias":
    col1, col2 = st.columns([1.5, 1])

    # Columna de la izquierda: listado de tickets
    with col1:
        st.subheader("Listado de Tickets")

        # Botones de filtrado
        filtro = st.radio("Filtrar por estado:", ["Todos", "Abiertos (OPEN)"], horizontal=True, label_visibility="collapsed")
        # Llamada a service para recuperar los tickets que cumplen el filtro
        tickets = service.get_all_tickets() if filtro == "Todos" else service.get_open_tickets()
        # Si hay tickets
        if tickets:
            # Desempaquetado y creación de diccionario válido para Streamlit
            datos_tickets = [{**t.__dict__, "status": t.status.value} for t in tickets]
            # Pintado de los datos como tabla
            st.dataframe(datos_tickets, width="stretch")
        else:
            st.info("No hay tickets que mostrar.")
```

Bloque de código encargado de mostrar y gestionar la tabla principal de **incidencias** en la parte izquierda de la pantalla. Combina **diseño** de interfaz, **lógica** condicional y una **transformación de datos muy importante**.

> Ubicación y Título

```
with col1:
    st.subheader("Listado de Tickets")
```

Al usar **with col1**: se le indica a **Streamlit** que todo lo programado a continuación se dibuja exclusivamente dentro de la columna izquierda, definida previamente. Seguidamente se imprime el subtítulo de la sección.

> Filtro Interactivo >

```
filtro = st.radio("Filtrar por estado:", ["Todos", "Abiertos (OPEN)"], horizontal=True, label_visibility="collapsed")
```

Se crean dos botones redondos de selección (**st.radio**) para que el usuario pueda elegir qué quiere ver, filtrando por estado. **horizontal=True**: hace que los botones se coloquen uno al lado del otro en lugar de apilarse, ahorrando espacio vertical. **label_visibility="collapsed"**: es un pequeño truco de diseño; **oculta el texto principal "Filtrar por estado:"** para que la interfaz quede más limpia, dejando solo las opciones visibles. El **resultado** ("Todos" o "Abiertos (OPEN)") se guarda en la variable **filtro**.

> Consulta Inteligente (Regla de Negocio) >

```
tickets = service.get_all_tickets() if filtro == "Todos" else service.get_open_tickets()
```

Se emplea un **"operador ternario"** de Python. Es un **if/else escrito en una sola línea** para que sea más elegante. Le pide a la capa de servicios ("**service.py**") que devuelva la lista completa de tickets en caso de seleccionarse la opción **'Todos'**; caso contrario ejecuta la consulta que trae solo los que están en estado **OPEN** y guarda el resultado en la variable **tickets**.

>> Plan A (existen tickets en la base de datos)

Si hay tickets en la base de datos, entra en este **if**.

> **Transformación de los Datos** >

if tickets:

`datos_tickets = [{**t.__dict__, "status": t.status.value} for t in tickets]`

La base de datos y el servicio devuelven una **lista de objetos de la clase Ticket**. Sin embargo, las **tablas** de **Streamlit** (que por debajo usan una tecnología llamada **PyArrow**) a veces **se bloquean** si se intentan meter **objetos complejos** o **Enums** (enumeradores). Para evitar errores, se emplea comprensión de listas:

- `t.__dict__`: convierte automáticamente el objeto **Ticket** en un **diccionario estándar de Python** que **Streamlit** puede leer fácilmente.
- `"status": t.status.value`: sobrescribe específicamente el campo del estado. Como el estado es un Enum (ej: TicketStatus.OPEN), se extrae su **.value** (el texto simple "OPEN") para que la tabla lo muestre correctamente sin dar errores.

`{**t.__dict__}` > **desempaquetado de diccionarios**.

**** (El operador de desempaquetado)**

Los dos asteriscos indican a Python que coja el diccionario a su derecha, lo abra y saque todos los pares de datos que hay dentro.

t (El objeto)

En el código, t representa un Ticket individual que se ha obtenido de la base de datos (por ejemplo, el ticket número 1 de "Fallo en red").

.__dict__ (El diccionario interno)

En Python, casi todos los objetos tienen un **atributo secreto llamado __dict__**, que coge todas las variables de ese objeto y las convierte instantáneamente en un diccionario.

Dentro de t.__dict__, se vería algo así:

```
{
  "id": 1,
  "title": "Fallo en red",
  "description": "Pérdida de paquetes",
  "status": <TicketStatus.OPEN: 'OPEN'> # ¡Aquí está el problema!
}
```

{ } (Las llaves del nuevo diccionario)

Las llaves exteriores crean un diccionario totalmente nuevo.

```
{**t.__dict__, "status": t.status.value}
```

> Esta línea tiene por objeto **solucionar** un **problema** técnico con las **tablas visuales** de Streamlit.

Streamlit se bloquea si se intenta meter un Enum (como <TicketStatus.OPEN>) en sus tablas; solo acepta texto **normal**, **números** y **booleanos**.

Con esta línea se está haciendo un "clon manipulado" del ticket siguiendo estos pasos:

- Crea un diccionario vacío: {
- Vuelca todos los datos originales del ticket dentro: `**t.__dict__`
- Sobrescribe el estado: antes de cerrar la llave, `"status": t.status.value` busca la variable **"status"** recién volcada y la sobrescribe, sustituyendo el objeto Enum complejo por el texto plano "OPEN".
- Cierra el diccionario: }

> De una se copian todos los datos del **ticket** y se "limpia" el estado para que **Streamlit** pueda dibujarlo en la tabla sin que la aplicación se cuelgue. Todo en una sola línea de código en lugar de tener que escribir un bucle largo.

> El Dibujado de la Tabla > `st.dataframe(datos_tickets, width="stretch")`:

Una vez que los datos están "limpios" y traducidos a diccionarios, la función ***st.dataframe*** los toma y dibuja la tabla interactiva en la pantalla. El parámetro ***width="stretch"*** asegura que la tabla se expanda horizontalmente para rellenar todo el ancho disponible de la columna 1, dándole un aspecto sólido y equilibrado.

>> **Plan B (*Interfaz Vacía*)**

Si no hay tickets en la base de datos, entra en este **else**.

else:

st.info("No hay tickets que mostrar.")

Si la base de datos está vacía o si el filtro aplicado no devuelve ningún resultado (por ejemplo, si no hay tickets abiertos), este **else** evita que se dibuje una tabla vacía y fea. En su lugar, muestra un elegante recuadro azul informativo avisando al usuario de que no hay datos.

```

with col2:
    st.subheader(" + Nuevo Ticket")

    # Formulario
    with st.form("form_nuevo_ticket", clear_on_submit=True):
        titulo = st.text_input("Título de la incidencia")
        desc = st.text_area("Descripción detallada")
        email = st.text_input("Email del solicitante")

    # Botón de envío del formulario
    submit_ticket = st.form_submit_button("Crear Ticket", type="primary")

    # Al clicar el botón de envío
    if submit_ticket:
        try:
            # Llamada a service para crear un nuevo ticket
            nuevo_ticket = service.create_ticket(titulo, desc, email)
            st.success(f"Ticket #{nuevo_ticket.id} creado con éxito.")
            st.rerun()
        # Manejo de error en caso de algún formato de dato no válido
        except ValueError as e:
            st.error(str(e))

```

Bloque de código responsable de **capturar la información del usuario y enviarla de forma segura a la base de datos** para **registrar una nueva incidencia**. A diferencia de la tabla anterior, que solo lee datos, este bloque escribe datos, por lo que utiliza una estructura llamada Formulario (**form**).

> Ubicación y Título >

```

with col2:
    st.subheader(" + Nuevo Ticket")

```

Como anteriormente, se emplea **with col2:** para indicar a **Streamlit** que todo este bloque debe dibujarse en la columna derecha (la más estrecha). Acto seguido, se imprime el título de esta sección.

> Creación del Formulario (**st.form**) > **with st.form("form_nuevo_ticket", clear_on_submit=True):**

- **Agrupación:** un formulario en **Streamlit** empaqueta todos los campos de texto y el botón en un solo bloque visual. Hasta que el usuario no pulsa el botón final, **Streamlit** no envía nada al servidor, lo que ahorra recursos. Cada formulario necesita un identificador único, en este caso **"form_nuevo_ticket"**.
- **Limpieza automática:** el parámetro **clear_on_submit=True** es un detalle de experiencia de usuario (**UX**), que indica al sistema que borre la información de las cajas de texto si la creación es exitosa, dejando el formulario en blanco y listo para el siguiente **ticket**.

> Campos de Entrada:

```

titulo = st.text_input("Título de la incidencia")
desc = st.text_area("Descripción detallada")
email = st.text_input("Email del solicitante")

```

Se definen las cajas donde el usuario escribirá.

- **st.text_input** crea una caja de texto de una sola línea (ideal para títulos o emails).
- **st.text_area** crea un **cuadro de texto** más grande y multilínea (necesario para explicaciones largas como la descripción del problema).

Lo que el usuario escriba en cada caja se guardará temporalmente en las variables **titulo**, **desc** y **email**.

> Botón de Envío > **submit_ticket = st.form_submit_button("Crear Ticket", type="primary"):**

A diferencia de un botón normal de **Streamlit**, los formularios requieren obligatoriamente un **st.form_submit_button**. El parámetro **type="primary"** se emplea para indicar al archivo **style.css** que pinte este botón de color azul destacado para llamar la atención del usuario. Si el botón es pulsado, la variable **submit_ticket** se vuelve **True**.

> Ejecución y Validación:

```
if submit_ticket:
    try:
        nuevo_ticket = service.create_ticket(titulo, desc, email)
        st.success(f"Ticket #{nuevo_ticket.id} creado con éxito.")
        st.rerun()
    except ValueError as e:
        st.error(str(e))
```

Si el usuario **pulsó el botón**, entramos en este bloque condicional > **try**:

Se intenta llamar a la regla de negocio (**service.create_ticket**) pasándole los textos que el usuario escribió.

Si el servicio aprueba los datos (por ejemplo, verifica que el email tiene una "@") y guarda el **ticket** en la base de datos, se muestra un recuadro verde de éxito (**st.success**) y se ejecuta **st.rerun()**, lo que reinicia la pantalla entera para que la tabla de la columna 1 se actualice al instante y muestre el **ticket** recién creado.

En caso de error (**except**), si el usuario dejó el título en blanco o puso un email sin "@", el archivo "**service.py**" lanzará un error **ValueError**. Este bloque lo atrapa antes de que la aplicación se rompa y muestra el mensaje de error en un recuadro rojo (**st.error**) para que el usuario corrija su equivocación.

```

# -----
# PESTAÑA 2: TÉCNICOS
# -----
elif selected == "Técnicos":
    col1, col2 = st.columns([1.5, 1])

    # Columna de la izquierda: listado de técnicos y apartado de bajas
    with col1:
        st.subheader("🛠️ Plantilla de Técnicos")
        # Llamada al service para recuperar todos los técnicos de la BBDD
        tecnicos = service.get_all_technician()

        # Si hay técnicos
        if tecnicos:
            # Conversión a diccionario válido para Streamlit y muestra en tabla
            st.dataframe([t.__dict__ for t in tecnicos], width="stretch")

            # SECCIÓN DE BAJA DE TÉCNICO
            st.markdown("#### ❌ Dar de baja a un técnico")
            # Nueva lista con "solo técnicos en activo"
            tecnicos_activos = [t for t in tecnicos if t.active]

            # Si hay técnicos en activo
            if tecnicos_activos:
                # Creación de un formulario
                with st.form("form_baja_tecnico"):
                    # Selector desplegable
                    # Almacena el ID de la selección
                    # aunque en pantalla muestra el nombre (gracias a la función lambda)
                    tecnico_id_baja = st.selectbox(
                        "Selecciona el técnico a desactivar:",
                        [t.id for t in tecnicos_activos],
                        format_func=lambda x: next(t.name for t in tecnicos_activos if t.id == x)
                    )

                    # Botón de envío de formulario para dar de baja
                    if st.form_submit_button("Dar de Baja", type="primary"):
                        nombre_tecnico = next(t.name for t in tecnicos_activos if t.id == tecnico_id_baja)
                        # Se pasa el ID y el nombre del técnico a la ventana modal programada anteriormente
                        dialog_confirmar_baja(tecnico_id_baja, nombre_tecnico)

            # Si NO hay técnicos en activo
            else:
                st.info("No hay técnicos activos actualmente.")
        # Si NO hay técnicos
        else:
            st.info("No hay técnicos registrados.")

```

Este bloque de código controla la mitad izquierda de la pestaña "Técnicos". Muestra la **plantilla completa** y, justo debajo, proporcionar un **mecanismo** seguro para **dar de baja** a un empleado, conteniendo una mezcla de **diseño web** y **lógica** de Python:

> Entrada y Estructura Visual

```

elif selected == "Técnicos":
    col1, col2 = st.columns([1.5, 1])
    with col1:

```

elif: introduce el código a ejecutar en caso de selección de la pestaña 'Técnicos' en el menú superior.

st.columns([1.5, 1]): divide la pantalla en dos. Se le da un tamaño de 1.5 a la primera columna para que sea un poco más ancha que la segunda, dándole más espacio a la tabla de datos.

Con **with col1:**, se da acceso al interior de la columna izquierda.

> Consulta y Visualización de la Tabla

```
st.subheader("👤 Plantilla de Técnicos")
tecnicos = service.get_all_technician()
```

```
if tecnicos:
```

```
    st.dataframe([t.__dict__ for t in tecnicos], width="stretch")
```

Se pide a la capa de servicios ("**service.py**") que recupere la lista completa de todos los técnicos que existen en la base de datos (tanto los activos como los inactivos).

Si la lista no está vacía (**if tecnicos:**), dibuja la tabla interactiva (**st.dataframe**).

Aquí vuelve a usarse **t.__dict__**, pero dentro de unos corchetes [...]. Se emplea una "**comprensión de listas**": en una sola línea, convierte cada objeto **Technician** en un diccionario apto para la tabla.

> Filtro (Solo Activos)

```
st.markdown("#### ❌ Dar de baja a un técnico")
tecnicos_activos = [t for t in tecnicos if t.active]
```

Antes de dibujar el selector para dar de baja a alguien se aplica la lógica pura: no se puede dar de baja a alguien que ya está de baja. La línea filtra la lista original de técnicos y crea una lista nueva (**tecnicos_activos**) guardando únicamente a aquellos cuya propiedad **active** sea igual a 1 (True).

> Selector (st.selectbox)

```
if tecnicos_activos:
    with st.form("form_baja_tecnico"):
        tecnico_id_baja = st.selectbox(
            "Selecciona el técnico a desactivar:",
            [t.id for t in tecnicos_activos],
            format_func=lambda x: next(t.name for t in tecnicos_activos if t.id == x)
        )
```

Si hay técnicos activos se abre un formulario con un desplegable (**st.selectbox**).

En la lista de técnicos activos se encuentran los IDs numéricos (1, 2, 3...) de cada uno, pero interesan los nombres ("Ana", "Luis"):

- Se pasan como opciones los **ID**: **[t.id for t in tecnicos_activos]**, toma el ID de cada técnico de la lista.
- Seguidamente, se encuentra el traductor (**format_func**): esta función (**lambda x**) actúa como una **máscara visual**. Toma el número **ID** que Python está manejando por debajo, busca en la lista de técnicos activos cuál coincide con ese ID, y muestra su **nombre** en la pantalla.
El usuario lee, por ejemplo, "Ana García", pero cuando hace clic, el programa guarda internamente el ID 1 en la variable **tecnico_id_baja**.

format_func=lambda x: next(t.name for t in tecnicos_activos if t.id == x)

Condensa lo que normalmente ocuparían 5 o 6 líneas de código en una sola.

Su objetivo principal es hacer de traductor simultáneo entre la máquina (que entiende de números/IDs) y el usuario humano (que entiende de nombres).

> **st.selectbox**:

- Está configurado para manejar internamente los IDs de los técnicos ([1, 2, 3]).
- Parámetros > (label, options, index=0, format_func=special_internal_function):

```
label >          "Selecciona el técnico a desactivar:",
options >        [t.id for t in tecnicos_activos],
format_func >    lambda x: next(t.name for t in tecnicos_activos if t.id == x)
```

format_func: es un parámetro **nativo** de **Streamlit** que indica al desplegable que antes de dibujar cada opción en pantalla la pase por esta función; se le pasa una función para modificar las opciones que se muestran. Recibe como argumento las opciones en crudo.

lambda x: es una forma rápida de crear una **función "anónima"** de "usar" y "tirar" en Python. Aquí, la **x** representa el **ID** numérico que **Streamlit** está a punto de dibujar (por ejemplo, el ID 1).

> Buscador interno (**t.name for t in tecnicos_activos if t.id == x**)

Esta parte que está entre paréntesis se llama **expresión generadora**. Es un **mini bucle** de búsqueda.

Indica a Python que recorra la lista de **tecnicos_activos** y para cada técnico (**t**) que encuentre compruebe si su **ID** coincide con la **x** y en tal caso devuelva su nombre (**t.name**).

> **next(...)**

La función next() es un comando **nativo** de **Python** que indica que se **realice la devolución del primer resultado que se encuentre y se detenga inmediatamente**.

next() envuelve el mini bucle. En el instante exacto en que encuentra al técnico con el ID buscado, el next() se queda con su nombre y cancela la búsqueda al instante, de modo que no se siguen revisando el resto.

Es una forma extremadamente rápida y eficiente de buscar un único elemento exacto dentro de una lista.

Lambda

Para entender cómo **x** toma el valor del ID, tenemos que mirar quién está llamando a esa función lambda. Y la respuesta es: **Streamlit** lo hace por detrás.

```
st.selectbox(
    "Selecciona el técnico a desactivar:",      # 1. El título
    [t.id for t in tecnicos_activos],          # 2. LAS OPCIONES REALES
    format_func=lambda x: next(...)            # 3. La regla de formato
)
```

En la línea 2 (las **opciones**), se pasa explícitamente una lista de **IDs** matemáticos.

Cuando **Streamlit** va a **dibujar** la página web, mira la línea 3 (**format_func**) y entiende que no se quiere que imprima los números directamente en la pantalla; quiere que los pase por esta pequeña máquina traductora (el **lambda**).

Streamlit internamente hace un bucle automático:

- Coge el primer elemento de las opciones (el **ID 1**).
- Lo inyecta al **lambda**. En ese milisegundo, **x** vale **1**; el **lambda** busca y devuelve el nombre correspondiente.
- Coge el segundo elemento (el **ID 2**). Se lo inyecta al lambda y ahora **x** vale **2**...

La **x** es simplemente un embudo; adquiere el valor del ID porque **Streamlit** le está pasando los IDs de la lista de opciones uno por uno.

Se usa **lambda x**: simplemente para no tener que escribir una función entera (con su def y su return) para algo que solo se usa una vez en toda la aplicación. Y se llama a la variable **x** por pura costumbre.

> El Botón y la Ventana Modal

```
if st.form_submit_button("Dar de Baja", type="primary"):
    nombre_tecnico = next(t.name for t in tecnicos_activos if t.id == tecnico_id_baja)
    dialog_confirmar_baja(tecnico_id_baja, nombre_tecnico)
```

Si el usuario pulsa el botón principal, no se toca la base de datos de inmediato:

- Primero, **recupera el nombre exacto del técnico** seleccionado usando su ID mediante la función **next()**. Busca el primer técnico cuyo ID sea el seleccionado, se saca el nombre y se guarda en la variable **nombre_tecnico**.
- Posteriormente se invoca la **ventana modal** **dialog_confirmar_baja(...)** programada en la parte superior del archivo y se le pasa el **ID** y el **nombre**. En este punto, la pantalla se oscurece y el **pop-up** rojo toma el control absoluto para pedir la confirmación final.

> Restricciones de Seguridad

else:

st.info("No hay técnicos activos actualmente.")

else:

st.info("No hay técnicos registrados.")

Estos **else** al final son fundamentales para la experiencia de usuario. Si **no hay técnicos** en toda la empresa, **o** si los que hay ya están **todos dados de baja**, **el formulario de baja desaparece automáticamente** para no confundir al usuario, y en su lugar se muestra un mensaje informativo de color azul.

```
# Columna de la derecha: creación de un nuevo técnico
with col2:
    st.subheader(" + Nuevo Técnico")
    # Formulario
    with st.form("form_nuevo_tecnico", clear_on_submit=True):
        nombre = st.text_input("Nombre completo")
        email_tech = st.text_input("Correo electrónico")
        submit_tech = st.form_submit_button("Registrar Técnico", type="primary")

    # Si se clicca el botón de envío
    if submit_tech:
        try:
            # Llamada al service para registrar el nuevo técnico
            nuevo_tecnico = service.register_technician(nombre, email_tech)
            st.success(f"Técnico {nuevo_tecnico.name} registrado.")
            # Reinicio de la app
            st.rerun()
        # Manejo de errores o excepciones (reglas de negocio)
        except Exception as e:
            st.error(f"Error: {e}")
```

Mitad derecha de la pestaña "Técnicos".

Su objetivo es capturar los **datos de un nuevo empleado** y enviarlos de forma segura a la base de datos.

Estructuralmente casi idéntico al formulario de "Nuevo Ticket".

> Ubicación visual

```
with col2:
    st.subheader(" + Nuevo Técnico")
```

Se empaqueta todo este bloque en la columna derecha de la pantalla (un poco más estrecha que la de la tabla). Luego, se dibuja el título.

> "Caja del Formulario

```
with st.form("form_nuevo_tecnico", clear_on_submit=True):
```

Al igual que con los tickets, se agrupa todo dentro de un formulario (**st.form**).

El identificador "**form_nuevo_tecnico**" es obligatorio para que **Streamlit** no lo confunda con los otros formularios de la aplicación.

clear_on_submit=True: si el registro tiene éxito, vacía automáticamente las cajas de texto para poder registrar a otro técnico después.

> Captura de Datos y Botón

```
nombre = st.text_input("Nombre completo")
email_tech = st.text_input("Correo electrónico")
submit_tech = st.form_submit_button("Registrar Técnico", type="primary")
```

Se crean dos **cajas de texto** simples (**st.text_input**) y se guarda lo que el usuario escriba en las variables **nombre** y **email_tech**. Se añade el **botón** obligatorio del formulario. El atributo **type="primary"** se conecta con el **CSS** para darle ese color destacado (azul fuerte) que le indica al usuario que es la acción principal.

> Lógica de Guardado

```
if submit_tech:
    try:
        nuevo_tecnico = service.register_technician(nombre, email_tech)
        st.success(f"Técnico {nuevo_tecnico.name} registrado.")
        st.rerun()
```

Cuando el usuario hace clic, entramos en el bloque if y se llama a la capa de servicios (**service.register_technician**)0000.

Si el servicio valida todo correctamente, se guarda en **SQLite** y devuelve el **objeto** del nuevo técnico; además se mostraría un mensaje verde de éxito (**st.success**).

Finalmente se ejecuta **st.rerun()** que reinicia la pantalla entera y se **actualiza la tabla** de la columna 1 vuelve al consultar la base de datos sin que el usuario tenga que darle a F5.

> Manejo de Errores

*except Exception as e:**st.error(f"Error: {e}")*

En el archivo *"service.py"* se programaron reglas de negocio estrictas: no permitir nombres en blanco, exigir que el email tenga una "@", o evitar que se repitan correos electrónicos.

Si el usuario rompe alguna de esas reglas, el *"service.py"* lanza una **alarma**. Este bloque **except** atrapa esa alarma. En lugar de mostrar una pantalla roja de sistema colgado, extrae el texto del error (la variable *e*) y lo dibuja en un recuadro rojo (*st.error*) para que el usuario sepa exactamente qué tiene que corregir.

```
# -----
# PESTAÑA 3: GESTIÓN Y ASIGNACIÓN
# -----
elif selected == "Gestión y Asignación":
    st.subheader("⚙️ Acciones Operativas")

    # Llamada al service para extraer todos los tickets y todos los técnicos de la BBDD
    todos_tickets = service.get_all_tickets()
    todos_tecnicos = service.get_all_technician()

    # Nueva lista con "solo los técnicos en activo"
    tecnicos_activos = [t for t in todos_tecnicos if t.active]
```

Este bloque de código es la **"fase de preparación"** para la tercera pestaña. A diferencia de las pestañas anteriores donde se dividía la pantalla en dos columnas en primer lugar, aquí lo primero que se hace es **recolectar todos los datos necesarios para las operaciones más complejas**.

> Enrutador > *elif selected == "Gestión y Asignación":*

Si la variable **selected** coincide exactamente con este texto, Python sabe que el usuario hizo clic en la tercera pestaña y le permite el paso a este bloque de código.

> Título de la Sección > *st.subheader("⚙️ Acciones Operativas")*

Simplemente dibuja el título principal de esta pestaña usando la función nativa de **Streamlit** para subtítulos (**<h2>** en **HTML**).

> Recolección Masiva de Datos

```
todos_tickets = service.get_all_tickets()
todos_tecnicos = service.get_all_technician()
```

Antes de poder asignar un ticket a un técnico o cambiar su estado, es necesario conocer qué hay en la base de datos.

Se llama a la capa *"service.py"* dos veces:

- En la variable **todos_tickets** se vuelcan todos los tickets que existen (abiertos, en progreso y cerrados).
- En **todos_tecnicos** se vuelca toda la plantilla histórica (activos y dados de baja).

Tener todo esto cargado en la memoria desde el principio de la pestaña hace que los formularios que se dibujan más abajo funcionen mucho más rápidos, ya que no tienen que estar preguntando a la base de datos constantemente.

> Filtro de Regla de Negocio

```
tecnicos_activos = [t for t in todos_tecnicos if t.active]
```

Es una comprensión de listas que aplica una regla de negocio fundamental: **no puede asignarse trabajo a un empleado que ya no está en la empresa**.

Se recorre la lista **todos_tecnicos** y extraer únicamente a aquellos cuyo estado **active** sea verdadero (**True** o **1**). Esa nueva lista filtrada se guarda en **tecnicos_activos**, y será la que se utilice más adelante para rellenar los desplegables de **"Seleccionar técnico"**, garantizando que nunca se le asigne un ticket a alguien dado de baja por error.

Caso de haber tickets y técnicos en la BBDD
if todos_tickets and tecnicos_activos:

```
# Panel de Asignación
with col_asig:
    # Formulario
    with st.form("form_asignar_ticket"):
        st.markdown("#### 🛠️ Asignar Técnico a Ticket")
        # Desplegable para seleccionar el ticket a asignar
        ticket_id_asignar = st.selectbox("Selecciona Ticket:", [t.id for t in todos_tickets])
        # Desplegable para seleccionar el técnico a asignar
        # (mediante ID, pero mostrando en pantalla el nombre)
        tecnico_id_asignar = st.selectbox(
            "Selecciona Técnico:",
            [t.id for t in tecnicos_activos],
            format_func=lambda x: next(t.name for t in tecnicos_activos if t.id == x)
        )

        # Botón de envío
        if st.form_submit_button("Asignar Ticket", type="primary"):
            try:
                # Llamada al service para la asignación
                service.assign_ticket(ticket_id_asignar, tecnico_id_asignar)
                st.success("¡Asignación realizada con éxito!")
                st.rerun() # Reinicio de la app
            except ValueError as e:
                st.warning(str(e))
```

Este bloque de código se encarga de **emparejar** un problema (**ticket**) con un solucionador (**técnico**) y de **actualizar el estado** de ese ticket automáticamente.

> Ubicación y Empaquetado

```
# Panel de Asignación
with col_asig:
    with st.form("form_asignar_ticket"):
        st.markdown("#### 🛠️ Asignar Técnico a Ticket")
```

with col_asig: esta pestaña se divide en varias columnas; aquí se indica a **Streamlit** que dibuje este formulario en la columna asignada para esta tarea (**col_asig**).

with st.form(...): se vuelve a crear el envoltorio de un formulario; es vital aquí, para que la app no intente hacer la asignación en el momento en que el usuario toca el primer desplegable; debe esperar a que pulse el botón azul de confirmación.

> Desplegables de Selección

```
ticket_id_asignar = st.selectbox("Selecciona Ticket:", [t.id for t in todos_tickets])

tecnico_id_asignar = st.selectbox(
    "Selecciona Técnico:",
    [t.id for t in tecnicos_activos],
    format_func=lambda x: next(t.name for t in tecnicos_activos if t.id == x)
)
```

Aquí hay 2 selectores (**st.selectbox**), pero funcionan de forma ligeramente distinta:

- El **selector de Tickets**: toma la lista **todos_tickets** y extrae solo los **IDs** ([t.id for...]). Como los tickets normalmente se identifican por su número (ej. Ticket #1, Ticket #2), aquí no se hace ninguna traducción, se le muestran los números directamente al usuario.
- El **selector de Técnicos**: se usa la lista filtrada **tecnicos_activos** para no asignar trabajo a técnicos dada de baja y se vuelve a emplear el **traductor simultáneo**: **format_func=lambda x: next(...)** de modo que internamente **Streamlit** maneja el ID (1, 2, 3), pero el usuario lee el nombre ("Ana", "Luis").

> Ejecución de la Regla de Negocio

```
if st.form_submit_button("Asignar Ticket", type="primary"):
    try:
        service.assign_ticket(ticket_id_asignar, tecnico_id_asignar)
        st.success("¡Asignación realizada con éxito!")
    st.rerun()
```

Cuando el usuario pulsa el botón, el código intenta hacer la operación:

- Llama a `service.assign_ticket(...)` pasándole los dos números (el ID del **ticket** y el ID del **técnico**).
Revisando “`service.py`” se podría observar que esta función no solo vincula el técnico al ticket, sino que además cambia automáticamente el **estado** del ticket de OPEN a **IN_PROGRESS**.
- Si la base de datos lo guarda correctamente, muestra un mensaje verde y lanza un `st.rerun()` para actualizar toda la pantalla al instante.

> Errores

```
except ValueError as e:
    st.warning(str(e))
```

Si el usuario intenta asignar un **ticket** que ya estaba cerrado (**CLOSED**) la capa de servicios (“`service.py`”) no se lo va a permitir y lanzará un error.

Este bloque **except** atrapa ese error y, en lugar de romper el programa, muestra una **advertencia en color amarillo** (`st.warning`) con el mensaje exacto programado en el servicio, explicando por qué no se puede realizar esa asignación.

```

# Panel de Cierre
with col_cierre:
    # Formulario
    with st.form("form_cerrar_ticket"):
        st.markdown("#### 🛑 Cerrar Ticket")
        # Desplegable para selección de ticket
        ticket_id_cerrar = st.selectbox("Selecciona Ticket a cerrar:", [t.id for t in todos_tickets])

    # Si se clicka el botón
    if st.form_submit_button("Cerrar Ticket", type="primary"):
        try:
            # Llamada al service para cerrar el ticket
            service.close_ticket(ticket_id_cerrar)
            st.success("¡Ticket cerrado correctamente!")
            st.rerun() # Reinicio de la app
        except ValueError as e:
            st.error(str(e))

```

El bloque gestiona la última fase del ciclo de vida de una incidencia: el **cierre del ticket**. Su estructura es muy similar a la de asignación.

> Ubicación y Formulario

```

with col_cierre:
    with st.form("form_cerrar_ticket"):
        st.markdown("#### 🛑 Cerrar Ticket")

```

with col_cierre: coloca todo este bloque en la columna específica preparada para las tareas de cierre.

with st.form("form_cerrar_ticket"): vuelve a usar una **caja** para agrupar los elementos **selector** y **botón** para que el programa espere a que el usuario confirme una decisión antes de enviar cualquier orden a la base de datos.

> Selector de Tickets > `ticket_id_cerrar = st.selectbox("Selecciona Ticket a cerrar:", [t.id for t in todos_tickets])`

Crea un desplegable simple (**st.selectbox**).

Utiliza la lista **todos_tickets** cargada al principio de la pestaña y extrae de ella únicamente los números de **ID** ([t.id for t in todos_tickets]).

El número que el usuario elija en la pantalla quedará guardado temporalmente en la variable **ticket_id_cerrar**.

> Ejecución (Regla de Negocio)

```

if st.form_submit_button("Cerrar Ticket", type="primary"):
    try:
        service.close_ticket(ticket_id_cerrar)
        st.success("¡Ticket cerrado correctamente!")
        st.rerun()

```

Cuando el usuario pulsa el botón principal:

- El código llama a la función **close_ticket** de la capa de servicios ("**service.py**"), enviándole el **ID** del **ticket** seleccionado.
- El servicio busca ese ticket en la base de datos y cambiará su estado a **CLOSED**.
- Si la base de datos acepta el cambio sin problemas, la interfaz dibuja el recuadro verde de éxito (**st.success**) y ejecuta **st.rerun()**. Ese **st.rerun()** reinicia la pantalla entera para que si se vuelve a la pestaña "Ver Incidencias", el ticket ya aparezca con su nuevo estado actualizado.

> Manejo de errores

```

except ValueError as e:
    st.error(str(e))

```

Si un usuario intenta cerrar un **ticket ya cerrado** previamente, el archivo "**service.py**" lanzará un error (**raise ValueError(...)**).

Este bloque **except** atrapa ese error técnico, evita que la pantalla se congele con código indescifrable, y le muestra al usuario un educado **recuadro rojo** (**st.error**) con la advertencia exacta.

```

else:
    st.warning("Debe haber al menos un ticket y un técnico registrados para realizar asignaciones.")

```

A-2. style.css

```
/* =====
1. FONDO GENERAL Y LIMPIEZA
===== */
/* #root: contenedor principal
.stApp: caja gigante donde Streamlit dibuja la app */
html body div#root div.stApp {
  background-color: #c8d6e5 !important;
}
```

Este primer bloque **prepara el "lienzo"** de la aplicación. **Streamlit** tiene un diseño por defecto muy marcado (generalmente blanco o negro con menús predeterminados); aquí se le **obliga a cambiar sus reglas básicas** para que parezca un programa profesional y personalizado.

> Fondo General (El Lienzo)

```
html body div#root div.stApp {
  background-color: #c8d6e5 !important;
}
```

html body div#root div.stApp > le indica al navegador que entre en la página (**html**), vaya al cuerpo (**body**), busque el contenedor principal (**#root**) y apunte directamente a la caja gigante donde **Streamlit** dibuja la aplicación (**.stApp**). Es una ruta tan específica que no deja lugar a errores.

background-color: #c8d6e5: pinta el fondo de toda la pantalla de un color azul/grisáceo muy suave.

!important: **Streamlit** intenta imponer sus propios colores. Al añadir **!important** al final se le da a la orden prioridad absoluta, indicando al navegador que no importa qué reglas traiga **Streamlit** de fábrica.

```
/* Ocultar la barra superior (Deploy, menú) */

/* header[data-testid="stHeader"]: apunta a la barra superior que Streamlit genera por defecto. */
html body div#root div.stApp header[data-testid="stHeader"] {
  display: none !important;
  height: 0 !important;
  padding: 0 !important;
  margin: 0 !important;
}
```

> Limpieza de la Barra Superior

Con **Streamlit**, por defecto, siempre aparece una barra superior blanca flotante que tiene un botón de "**Deploy**" (Desplegar) y un **menú de tres puntitos** arriba a la derecha. Eso es útil cuando se está programando, pero cuando se entrega la app a un usuario final se ve feo y poco profesional.

header[data-testid="stHeader"]: apunta directamente a esa barra superior que **Streamlit** genera por defecto.

display: none !important: hace que esa **barra desaparezca** por completo de la pantalla; no la pinta de transparente, sino que la borra de la existencia visual.

height, padding, margin: 0: aunque el menú sea invisible, a veces los navegadores dejan el "**hueco**" vacío (una franja blanca de varios píxeles en la parte superior). Al poner la altura, los márgenes interiores (**padding**) y exteriores (**margin**) a **0** se garantiza que ese hueco desaparezca totalmente, permitiendo que el título gigante de "🚧 SISTEMA DE GESTIÓN..." suba hasta arriba del todo y encaje perfectamente en la pantalla.


```

/* =====
2. CONTENEDOR CENTRAL
===== */
/* El contenedor central toma un nombre u otro según la versión:
    block-container
    stMainBlockContainer */
html body div#root div.stApp div.main div.block-container,
html body div#root div.stApp div[data-testid="stMainBlockContainer"] {
    background-color: transparent !important;
    box-shadow: none !important;
    border: none !important;
    padding: 1.5rem 3rem 3rem 3rem !important;
    margin-top: -0.5rem !important; /* Se empuja arriba para llenar el hueco dejado */
    max-width: none !important;
}

```

Este bloque de código se encarga de modificar el **gran contenedor central** donde **Streamlit** coloca todos los textos, botones y tablas.

Por defecto, Streamlit encierra todo el contenido en una columna central bastante estrecha y rígida. Con este bloque CSS, se rompe esa jaula y preparamos el terreno para un diseño personalizado (la "carpeta" blanca).

> Selectores

```

html body div#root div.stApp div.main div.block-container,
html body div#root div.stApp div[data-testid="stMainBlockContainer"] { ... }

```

2 rutas separadas por una coma, porque **Streamlit** actualiza su código interno a menudo. (La **,** equivale a **O**). En algunas versiones, al contenedor central le llaman **.block-container**, y en otras versiones más modernas le ponen la etiqueta **data-testid="stMainBlockContainer"**.

Al poner las 2 rutas, se garantiza que el diseño no se rompa nunca, sin importar en qué versión de **Streamlit** o servidor se ejecute.

> Invisibilidad del Contenedor Base

```

background-color: transparent !important;
box-shadow: none !important;
border: none !important;

```

Se le quita al contenedor nativo de **Streamlit** cualquier color de fondo (**transparent**), cualquier sombra (**box-shadow: none**) y cualquier borde (**border: none**). Se hace este contenedor totalmente invisible para que el protagonismo se lo lleve el recuadro blanco fabricado más adelante con el identificador **#recuadro-principal**.

> Ajuste de Espacios

```

padding: 1.5rem 3rem 3rem 3rem !important;
margin-top: -0.5rem !important;

```

padding: margen interior. Sigue el orden de las agujas del reloj (Arriba, Derecha, Abajo, Izquierda). Se le da **1.5rem** (**rem** = root em = elemento raíz) arriba para que el título no se pegue al techo, y **3rem** en los lados para que el contenido tenga mucho espacio para "respirar" y no choque contra los bordes de la pantalla.

margin-top: con un valor negativo (**-0.5rem**) se está "tirando" literalmente de toda la aplicación hacia arriba. Esto compensa el hueco vacío que se dejó al borrar la barra superior en el paso anterior.

> Expansión Total > **max-width: none !important;**

Esta es la línea más importante de este bloque. Puesto que de fábrica **Streamlit limita el ancho de la app a unos 730px**, en una pantalla grande se verán enormes franjas vacías a los lados.

Con **max-width: none !important;** se rompe el límite y **se obliga a la aplicación a expandirse a lo ancho** para ocupar toda la pantalla disponible, aprovechando al máximo el monitor del usuario. Esto es vital para que la tabla de incidencias de la columna izquierda se vea ancha, cómoda y profesional.

```

/* =====
3. TÍTULO EN RECUADRO INDEPENDIENTE
===== */

/* En Python con Streamlit, la función st.title genera una etiqueta <h1>
Aquí se apunta a esa etiqueta <h1> */
html body div#root div.stApp h1 {
  color: #0f172a !important;
  font-weight: 700 !important;
  background-color: #f1f5f9 !important;
  border: 1px solid #64748b !important;
  padding: 1.5rem 2rem !important;
  border-radius: 8px !important;
  box-shadow: 0 4px 6px -1px rgba(0,0,0,0.05) !important;
  margin-bottom: 0.5rem !important;
  margin-top: 0.5rem !important;
}

```

Bloque que da formato al **título principal de la app** (**st.title**) a modo de una "tarjeta" o recuadro independiente de aspecto profesional.

En Python, con **Streamlit**, la función **st.title** generaba una etiqueta **<h1>**. Este código apunta a esa etiqueta y la transforma por completo.

> **Selector** > **html body div#root div.stApp h1 { ... }**

La misma ruta exacta vista anteriormente, pero esta vez termina en **h1**, indicando al navegador que busque el título principal y le aplique todo el código.

> **Tipografía**

color: #0f172a !important;
font-weight: 700 !important;

color: pinta las letras de un azul marino casi negro (#0f172a); color muy usado en paneles de administración modernos porque es más suave para la vista que el negro puro (#000000).

font-weight: 700: hace que la letra sea en negrita (**bold**) para darle presencia y jerarquía.

> **Efecto "Tarjeta" (Fondo y Bordes)**

background-color: #f1f5f9 !important;
border: 1px solid #64748b !important;
border-radius: 8px !important;

background-color: le da al título un fondo gris/azulado muy claro.

border: dibuja una línea sólida de **1px** de grosor, de color azul pizarra, para delimitar la caja.

border-radius: 8px: suaviza las 4 esquinas de la caja redondeándolas; es un estándar actual de diseño UI (Interfaz de Usuario) que hace que la aplicación se sienta más amigable y moderna.

> **Efecto 3D (Sombra)**

box-shadow: 0 4px 6px -1px rgba(0,0,0,0.05) !important;: aplica una sombra muy sutil debajo del recuadro.

rgba(0,0,0,0.05): es una sombra negra, pero con solo un **5%** de **opacidad** (casi transparente). Esto crea una ilusión óptica de profundidad, haciendo que el título parezca estar ligeramente "flotando" sobre el fondo azul general de la pantalla.

> **Márgenes y Rellenos**

padding: 1.5rem 2rem !important;
margin-bottom: 0.5rem !important;
margin-top: 0.5rem !important;

padding: espacio interior; da espacio entre el texto y el borde del recuadro para que las letras no se asfixien contra las líneas.

margin: espacio exterior; obliga a la caja a que mantenga una pequeña distancia de seguridad (**0.5rem**) tanto por arriba como por abajo, para no pegarse a la línea divisoria (<hr>) que viene justo después.

```
/* Líneas separadoras
Busca todas las líneas separadoras dentro de la app (.stApp) */
html body div#root div.stApp hr {
  border: none !important;
  height: 3px !important;
  background-color: #64748b !important;
  margin-top: 1.5rem !important;
  margin-bottom: 2.5rem !important;
}
```

Este bloque de código se encarga de estilizar las líneas horizontales de separación.

En **Streamlit**, cuando escribimos **st.divider()** o usamos **st.markdown("---")**, el sistema dibuja una línea horizontal para separar secciones. Sin embargo, las líneas que trae **Streamlit** por defecto son muy finas y débiles visualmente. Con este **CSS**, las convertimos en separadores robustos que estructuran la aplicación.

> **Selector hr** > **html body div#root div.stApp hr { ... }**

Esta regla indica al navegador que busque todas las líneas divisorias que dibuje **Streamlit** en la aplicación y se prepare para cambiarlas.

> **Borrón de estilo por defecto** > **border: none !important;**

Por defecto, los navegadores antiguos solían dibujar las líneas **hr** usando bordes con un efecto 3D hundido que hoy en día se ve muy anticuado. Con **border: none**, se borra ese estilo predeterminado para dejar la línea "limpia" y lista para ser pintada desde cero.

> **Nuevo Aspecto (Cuerpo y Color)**

height: 3px !important;
background-color: #64748b !important;

Le da un alto (**height**) de **3px**, lo que la hace tres veces más gruesa que una línea normal, dándole mucha más presencia. Luego, se pinta con **background-color: #64748b**, que es exactamente el mismo gris/azulado (pizarra) del borde del título, manteniendo así una coherencia visual en toda la aplicación.

> **Espacios (Asimetría intencionada)**

margin-top: 1.5rem !important;
margin-bottom: 2.5rem !important;

Se controla cuánto espacio la separa de los demás elementos; los valores son asimétricos:

- Un espacio normal por **arriba (1.5rem)**, separándola del elemento anterior.
- Un espacio mucho mayor por **abajo (2.5rem)**.
Esto es un truco de diseño (UX/UI). Al dejar más espacio por debajo de la línea se indica visualmente al cerebro del usuario la finalización definitiva de la sección; ayuda a que la interfaz no se vea amontonada.

Y todas las líneas llevan **!important** para que **Streamlit** pueda sobrescribir el diseño.

```

/* =====
4. RECUADRO GIGANTE DE CONTENIDO (La "Carpeta")
===== */
/* 1. Limpieza de cualquier margen residual del ancla invisible
element-container es el contenedor tipo que Streamlit pone a cada elemento
has() busca dentro del elemento el que tiene el marcador indicado #recuadro-principal */
div.element-container:has(#recuadro-principal) {
    display: none !important;
    margin: 0 !important;
    padding: 0 !important;
}

```

Cuando se escribe código en Python, por ejemplo, `st.button()`, *Streamlit*, por motivos de orden y diseño, **mete cada elemento dentro de una caja protectora invisible**. Esa caja se llama **element-container**.

El código HTML real que generaría Streamlit, sería algo así:

```

<div class="element-container">
  <button>Hacer algo</button>
</div>

```

En `"gestion_incidencias.py"` teníamos:

```

# =====
# 6. CONTENIDO DE LAS PESTAÑAS
# =====
with st.container():
    st.markdown("<div id='recuadro-principal'></div>", unsafe_allow_html=True)

```

Este código se empleaba para crear el recuadro blanco gigante (la "carpeta") donde está todo el contenido interactivo y lograr que se fusionara con las pestañas superiores. Como este contenedor no se puede nombrar, se inyectaba dentro un trozo de **HTML** puro: `<div id='recuadro-principal'></div>`, que es un elemento div (división) completamente vacío. El `unsafe_allow_html=True` hacía que *Streamlit* lo procesara como código web real.

Aunque el marcador `#recuadro-principal` está vacío y no ocupa espacio, la caja exterior (**element-container**) sí lo hace. Por defecto, *Streamlit* le pone a esas cajas un pequeño margen inferior (un hueco blanco) para que los textos y botones no se peguen unos a otros. Ese margen residual creaba un espacio en blanco muy feo que rompía el diseño de la interfaz.

La solución es este bloque de código en el **CSS**:

```

div.element-container:has(#recuadro-principal) {
    ...
}

```

Este bloque de CSS es el otro extremo que trabaja con el código del archivo .py; **busca** todas las **cajas** de embalaje (**div.element-container**), pero no las modifica (podría borrar toda la aplicación), sino que con el filtro `:has(...)` mira **dentro** de cada caja y aplica el diseño a las cajas que tengan dentro el marcador `#recuadro-principal` (la esconde con **display: none** y le quita los márgenes con **margin: 0**).

```

/* 2. Tiron del recuadro hacia arriba para cerrar el hueco
Busca el bloque vertical que contienen ese elemento contenedor a mover arriba */
div[data-testid="stVerticalBlock"]:has(> div.element-container #recuadro-principal) {
  background-color: #f1f5f9 !important;
  border: 1px solid #64748b !important;
  border-radius: 0 8px 8px 8px !important;
  padding: 2.5rem !important;
  box-shadow: 0 4px 6px -1px rgba(0,0,0,0.05) !important;
  position: relative !important;
  z-index: 1 !important;

  /* ¡MODIFICAR los px para mover la caja a la altura estéticamente adecuada! */
  margin-top: -38px !important;
}

```

> Localización del Contenedor Padre > `div[data-testid="stVerticalBlock"]:has(> div.element-container #recuadro-principal) {`

Se emplea el selector `:has()`.

Se indica al navegador web que rastree toda la página buscando un bloque vertical de Streamlit (`stVerticalBlock`). En particular, busca únicamente aquel que tenga el marcador `#recuadro-principal`. El CSS selecciona el `st.container()` de Python y se prepara para pintarlo.

¿Por qué se busca el `stVerticalBlock`?

En la app se pretende pintar de gris claro y poner bordes redondeados no a un texto suelto, ni a un botón, si no que se pretende crear una "carpeta" gigante (un recuadro contenedor) que envuelva y agrupe todos los elementos de esa pestaña (títulos, inputs, botones...).

En Python, al agrupar **elementos** (por ejemplo, al usar un `st.container()`, o simplemente cuando **Streamlit** pone varias cosas una debajo de otra en la pantalla principal o en una columna), **Streamlit** crea una **gran caja HTML invisible** para mantenerlos **unidos**; a esa gran caja contenedora, **Streamlit** la llama `stVerticalBlock` (porque apila las cosas verticalmente de arriba a abajo).

Al querer pintar esa gran caja gigante, el instinto normal en **CSS** es indicar que pinte el `stVerticalBlock`.

¿Cuál es el problema? Que **Streamlit** usa `stVerticalBlock` para TODO. La barra lateral es un bloque vertical, las columnas son bloques verticales, la página principal es un bloque vertical. De modo que, si se indica en el **CSS** que pinte todos los bloques verticales, la pantalla entera se volvería un caos de cajas superpuestas.

Con `div[data-testid="stVerticalBlock"]:has(> div.element-container #recuadro-principal) {` se filtra la caja a pintar.

> Pintado de la "Carpeta"

```

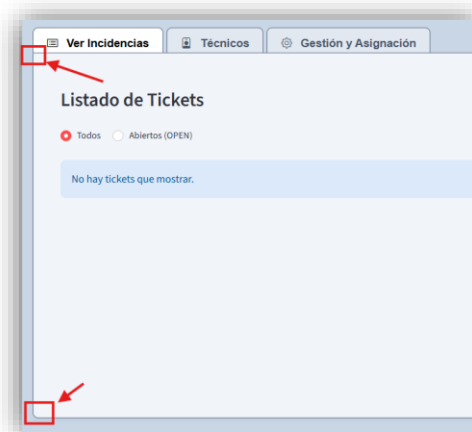
background-color: #f1f5f9 !important;
border: 1px solid #64748b !important;
border-radius: 0 8px 8px 8px !important;
padding: 2.5rem !important;
box-shadow: 0 4px 6px -1px rgba(0,0,0,0.05) !important;

```

Se le aplica el diseño corporativo:

Fondo y borde: mismo color gris claro y el mismo borde azul pizarra que el título.

border-radius: 0 8px 8px 8px: redondea todas las esquinas excepto la superior izquierda (0), puesto que es la que choca con la primera pestaña ("Ver Incidencias"). Al dejarla totalmente cuadrada y recta, da la sensación de que la pestaña y la caja son una sola pieza de cartulina recortada, como una carpeta de archivador físico.



> **Fusión pestañas-caja:****position: relative !important;****z-index: 1 !important;****margin-top: -38px !important;****}**

position y z-index: trata la caja como un elemento 3D y se coloca en la "Capa 1", preparándola para solaparse con el menú de arriba.

margin-top: -38px: como el **menú de navegación** de **Streamlit** dibuja un **margen** por debajo, la caja blanca quedaría separada de las pestañas flotando en el aire. Con este margen negativo **se lleva hacia arriba ocultando la línea divisoria** y creando la ilusión perfecta de que todo es un único elemento interactivo.

Desglose de **los 14 los contenedores verticales (stVerticalBlock)** que **Streamlit** genera en la aplicación:

> **Esqueleto**

- El contenedor principal (**Main**): la gran zona central donde se dibuja todo.
- La Barra Lateral (**Sidebar**): donde está configurado el menú de navegación para saltar entre secciones.
- El contenedor **st.container()** gigante donde se colocó el identificador **#recuadro-principal** para englobar la pantalla activa.

> Pestaña "Ver Incidencias"

- El **bloque** de la **pestaña** en sí: el contenedor padre que se activa al seleccionar esta opción en el menú.
- El **contenedor** de visualización: donde se renderiza la gran **tabla/dataframe** con todos los tickets cargados.

> Pestaña "Gestión y Asignación"

- El **bloque** de la **pestaña**: el contenedor padre de esta sección.
- Columna Izquierda (**col_asig**): el bloque vertical de la mitad izquierda.
- Formulario **form_asignar_ticket**: el contenedor interno de **Streamlit** que agrupa los selectores de ticket, técnico y el botón.
- Columna Derecha (**col_cierre**): el bloque vertical de la mitad derecha.
- Formulario **form_cerrar_ticket**: el contenedor interno que agrupa el selector de cierre y su botón.

> Pestaña "Técnicos"

- El **bloque** de la **pestaña**: el contenedor padre de esta sección.
- Columna Izquierda (**col1**): el bloque que agrupa el título y la tabla donde se muestran los técnicos existentes.
- Columna Derecha (**col2**): el bloque donde se prepara la creación de un técnico.
- Formulario **form_nuevo_tecnico**: el contenedor interno que apila el campo de nombre, el campo de especialidad y el botón de registro.

```
/* =====
5. INPUTS, BOTONES Y FORMULARIOS (Internos)
===== */
```

Una vez configurado el diseño de la pantalla, el fondo y la carpeta gigante, a través de este bloque de código se desarrolla el **diseño de los elementos con los que el usuario va a interactuar directamente**: los **formularios**, las **cajas de texto** y los **desplegables**. La finalidad es guiar la vista del usuario para que sepa exactamente dónde tiene que escribir.

```
/* Selecciona todos los formularios */
html body div#root div.stApp div[data-testid="stForm"] {
  border: 1px solid #cbd5e1 !important;
  border-radius: 10px !important;
  padding: 1.5rem !important;
  background-color: #e2e8f0 !important;
  box-shadow: 0 2px 4px rgba(0,0,0,0.05) !important;
}
```

> Contenedor del Formulario (**stForm**)

En el archivo de Python “*gestión_incidentes.py*” se usó **with st.form(...)** en numerosas ocasiones (para crear tickets, asignar técnicos, etc.).

html body div#root div.stApp div[data-testid="stForm"] permite apuntar a todos esos formularios.

background-color: #e2e8f0: conseguimos un contraste visual, estableciendo para la "carpeta" gigante un fondo gris muy clarito (#f1f5f9) y dando a los formularios un gris un tono más oscuro, creando un **efecto** visual de "caja dentro de una caja" y delimitando perfectamente dónde empieza y termina el formulario.

Además, se aplican **bordes redondeados** un poco más pronunciados (**10px**), un margen interior amplio (**padding: 1.5rem**) y una sombra muy sutil, eliminando la sensación de software antiguo y tosco.

```
/* Selecciona todas las cajas de texto */
html body div#root div.stApp .stTextInput input,
html body div#root div.stApp .stTextArea textarea,
html body div#root div.stApp .stSelectbox div[data-baseweb="select"] {
  border-radius: 6px !important;
  border: 1px solid #cbd5e1 !important;
  background-color: #ffffff !important;
}
```

> Cajas de Escritura y Desplegables

Los selectores apuntan al interior de las **cajas** donde el usuario teclea su **nombre**, su **email** o abre un **desplegable**.

background-color: #ffffff: blanco puro; es un principio básico de Experiencia de Usuario (**UX**). Con el fondo del formulario grisáceo, al pintar las cajas de escritura de blanco puro, estas resaltan y llaman a la acción. El cerebro del usuario sabe instintivamente que ese espacio en blanco es para escribir.

div[data-baseweb="select"]: esta ruta extraña se emplea para apuntar al **desplegable**, dado que **Streamlit**, por debajo, utiliza una **librería de diseño llamada Base Web** y es necesario apuntar a su clase específica para lograr cambiarle el color.

```
/* Resalta la casilla en que se está escribiendo */
html body div#root div.stApp .stTextInput input:focus,
html body div#root div.stApp .stTextArea textarea:focus {
  border-color: #3b82f6 !important;
  box-shadow: 0 0 1px #3b82f6 !important;
}
```

> Efecto "Foco" (Interacción en vivo)

:focus: en **CSS**, representa el momento exacto en el que el usuario hace clic dentro de una caja de texto y el cursor del ratón empieza a parpadear.

Al hacer clic se **cambia el color** del borde gris por un azul brillante (#3b82f6).

Con **box-shadow**, en lugar de proyectar la sombra hacia abajo, se proyecta un halo azul de **1px alrededor de toda la caja**.

Esto permite que el usuario nunca se pierda, y si está rellenando un formulario largo, sepa en todo momento en qué casilla exacta está tecleando.

```
/* Botones secundarios


```

El bloque se encarga de dar estilo a los **botones secundarios** de la aplicación. Los botones **principales** indicados mediante **type="primary"** en Python (los de enviar formularios) usan otra clase distinta, por lo que no se verán afectados por este gris.

> Selector > **html body div#root div.stApp div.stButton > button {**

div.stButton > button: **Streamlit** envuelve sus botones en una caja llamada **stButton***. El símbolo **>** indica al navegador que busque la caja **stButton**, pero no le aplique el **estilo** a la caja, sino directamente al **botón real** que está justo dentro.

* Pero cuando se usa **st.form_submit_button(...)** dentro de un formulario, **Streamlit** envuelve ese botón en una caja especial que internamente se llama **stFormSubmitButton**.

> **Aspecto Base (Reposo)**

```
background-color: #e2e8f0 !important;
color: #0f172a !important;
border: 1px solid #94a3b8 !important;
border-radius: 6px !important;
```

Fondo y Texto: fondo gris claro (#e2e8f0) con un texto en azul marino muy oscuro (#0f172a). Esto crea un contraste excelente, haciendo que el texto sea muy fácil de leer.

Bordes: gris un poco más oscuro (#94a3b8) para delimitarlo y se redondean las **esquinas a 6px**, como las cajas de texto en el paso anterior; esto mantiene la coherencia visual.

> **Fluidez (Animación UX)**

```
transition: all 0.2s ease !important;
}
```

Con esta propiedad **transition** cualquier cambio que sufra este botón (por ejemplo, cambiar de color) no será brusco ni instantáneo, sino que ocurrirá como una suave animación que dura **0.2 segundos** (0.2s ease).

all: se indica que se anime cualquier propiedad que vaya a cambiar (ya sea el fondo, el borde o el texto).

ease (se traduce como "suavidad" o "soltura"):

> Define la **curva de aceleración** o el ritmo de la animación:

- Empieza despacio (arranque **suave**).
- **Acelera** en la mitad de la transición.
- **Frena** progresivamente antes de terminar (parada **suave**).

> Como imita la inercia del mundo físico, el ojo humano percibe este cambio como algo muy fluido, elegante y natural.

> Otras opciones:

linear: movimiento; no hay aceleración ni frenado; resulta muy artificial.

ease-in: empieza muy lento y acaba rápido (como un objeto que empieza a caer).

ease-out: empieza muy rápido y frena al final (es muy popular en **UI modernas** porque da sensación de respuesta ultrarrápida al ratón).

ease-in-out: una versión más simétrica de ease (misma suavidad exacta al arrancar y al frenar).

> Interacción (Efecto Hover)

```
html body div#root div.stApp div.stButton > button:hover {
  background-color: #cbd5e1 !important;
  border-color: #64748b !important;
}
```

La pseudo-clase **:hover**: aplica cuando el cursor del ratón está flotando justo por encima del elemento.

Cambio visual: al pasar el ratón por encima del botón, este cambia automáticamente a un **gris un poco más oscuro** (#cbd5e1) y su **borde también se oscurece**.

Es un **principio fundamental de accesibilidad**. Le da al usuario una confirmación visual (un "feedback" táctil digital) de que el botón es interactivo y está listo para ser pulsado. Y gracias a la línea **transition** del paso anterior, este oscurecimiento ocurrirá de forma muy suave.

```
/* Botón primario (Formularios)
  Aplica a los botones del tipo formulario
  y a los marcados en el código Python como type="primary" */
html body div#root div.stApp div.stFormSubmitButton > button,
html body div#root div.stApp button[kind="primary"] {
  background-color: #3b82f6 !important;
  color: white !important;
  border: none !important;
  border-radius: 6px !important;
  font-weight: 500 !important;
}

html body div#root div.stApp div.stFormSubmitButton > button:hover,
html body div#root div.stApp button[kind="primary"]:hover {
  background-color: #2563eb !important;
  color: white !important;
}
```

Este bloque toma el **control de los botones principales** (Call to Action) de la aplicación, como "Crear Ticket", "Asignar Ticket" o "Registrar Técnico".

En diseño web, siempre debe haber una **jerarquía visual clara**. El botón que ejecuta la acción final debe destacar sobre todo lo demás. Se logra del siguiente modo:

> Selectores del botón principal

```
html body div#root div.stApp div.stFormSubmitButton > button,
html body div#root div.stApp button[kind="primary"] {
```

div.stFormSubmitButton > button: apunta directamente a los botones que **Streamlit** genera automáticamente cuando se usa **st.form_submit_button()** dentro de los formularios; **stFormSubmitButton** es la caja en que **Streamlit** mete los botones al usar **st.form_submit_button(...)**. Se buscan esos botones y se les aplica el formato (azul con letras en blanco).

La , equivale a un **O**:

- **button[kind="primary"]**: apunta a cualquier botón suelto por la aplicación al que se le haya dado el atributo **type="primary"** en el código de Python. A estos botones **Streamlit** les habrá pegado una etiqueta oculta en el código **HTML** que dice **kind="primary"**.
- Los corchetes **[...]** en CSS sirven para buscar atributos ocultos.
- De este modo se indica al navegador que busque por toda la página cualquier botón que tenga pegada la etiqueta **kind="primary"** y lo pinte también de azul.

> **Aspecto Base (Jerarquía Visual)**

```
background-color: #3b82f6 !important;
color: white !important;
border: none !important;
border-radius: 6px !important;
font-weight: 500 !important;
}
```

Aquí se construye el aspecto de "Acción Principal":

- **Contraste extremo:** fondo azul brillante y vibrante (#3b82f6) con texto en blanco puro (white). El ojo del usuario va a ir directamente hacia este botón en cuanto termine de rellenar el formulario.
- Estilo plano (**Flat Design**): sin bordes (**border: none**) para que sea un bloque de color sólido y limpio.

font-weight: 500 (un grosor de letra "**Medium**", tipografía fuerte). Es un poco más gruesa que el texto **normal (400)** pero sin llegar a ser **negrita total (700)**. Le da al texto del botón una presencia sólida y segura.

border-radius: 6px: se mantienen la coherencia (esquinas con la misma curvatura que las cajas de texto y los botones secundarios).

> **Interacción (Efecto Hover)**

```
html body div#root div.stApp div.stFormSubmitButton > button:hover,
html body div#root div.stApp button[kind="primary"]:hover {
  background-color: #2563eb !important;
  color: white !important;
}
```

Aquí se define qué pasa cuando el usuario pasa el ratón por encima (:**hover**):

- Se cambia el **fondo a un azul más oscuro y profundo** (#2563eb).
- Esta es una **regla de oro en el diseño web moderno**: al pasar el ratón por encima de un botón de un **color vivo**, el color debe **oscurecerse**, no aclararse.
- Se fuerza el color: **white !important**; para garantizar que el texto no cambie de color accidentalmente durante la interacción.

Recordatorio de porqué el empleo de la orden **!important**:

- Streamlit está diseñado para que los programadores de Python no tengan que saber nada de diseño web. Por eso, cuando se arranca la app, **Streamlit inyecta automáticamente cientos de reglas CSS de fábrica** (sus propios colores, tamaños y márgenes) directamente en el navegador del usuario final.
- El problema es que estas reglas por defecto se cargan en el último milisegundo y de una forma muy "agresiva". Aunque se escriba en el archivo simplemente "background-color: #f1f5f9;" y el navegador lo lea, milisegundos después el código interno de Streamlit lo sobrescribirá.

La solución es el uso de **!important**. Al añadir **!important** antes del punto y coma final de cada regla que se pretenda que prevalezca, se rompe la jerarquía; indica al navegador que no importa lo que Streamlit cargue después y cómo esté programado el componente original. Esta regla manda sobre todas las demás de forma absoluta.